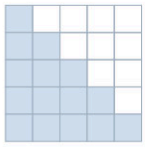
《数据结构与算法》第 5 章

数组与广义表

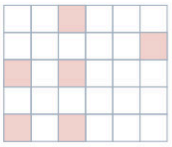
可视化基础学习讲义（C++ 版 · 复习用）

地址怎么算 · 特殊矩阵怎么压缩 · 稀疏矩阵怎么存 · 广义表怎么拆

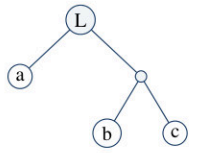




下三角：存 $\frac{n(n+1)}{2} + 1$ 个



稀疏：只记 (行, 列, 值)



L=(a,(b,c))，深度 2

配套 C++ 代码（4 个）

适用对象 第一次学习、或考前复习本章的本科生；全部代码均为 C++，并经 g++ 实际编译运行

范围约定 课件标注“不讲”的内容（十字链表、广义表存储结构、Array 类实现细节）只点到为止

目录

阅读说明 3

本章学什么、在课程里处于什么位置 3

学科小史与现实应用 4

原始资料重编码与纠错 5

第 1 章 数组：把“表格”排进一条线 7

1.1 数组的基本概念 7

1.2 数组的特点、基本操作与抽象数据类型 10

1.3 数组元素的存储顺序：行序与列序 11

1.4 元素存储位置的计算 12

第 2 章 矩阵的压缩存储：只存“有用的那部分” 19

2.1 矩阵与常规存储 19

2.2 对称矩阵 20

2.3 三角矩阵 24

2.4 三对角矩阵（对角矩阵） 28

2.5 稀疏矩阵 30

2.5.1 三元组顺序表 31

2.5.2 稀疏矩阵的转置 34

2.5.3 稀疏矩阵的相加与相乘 36

2.5.4 带行指针的链表 37

第 3 章 广义表：表中有表（只讲概念） 39

3.1 广义表的概念 39

3.2 表头、表尾与长度 40

3.3 广义表的深度 42

3.4 广义表的性质 44

3.5 广义表的抽象数据类型与基本操作 45

知识点—题目对应表 48

正式练习（38 题） 48

答案速查 **56**

详细解析 **56**

考前总复习 **71**

高频易错点总表 71
基础解题步骤总结 71
相近概念与跨学科 / 相似知识总对比 72
后续课程连接 72
随附 C++ 源文件 73
参考资料 73

阅读说明

这份讲义把课件、你的课堂笔记和课程 C++ 代码合在一起，重新按“先懂为什么、再记公式、最后会做题”的顺序组织。几个约定先说清楚，后面就不再重复：

1. **下标从几开始。**C++ 数组下标从 0 开始；课件讲特殊矩阵时，矩阵元素 a_{ij} 的 i, j 从 1 开始；压缩数组 sa 的下标有时从 0、有时从 1 开始。**这是本章最大的失分点**，每个公式旁边都会标明它用的是哪一种。
2. **颜色的含义。**浅蓝 = 定义、公式等核心知识；米色 = 例题、生活应用、历史与科普；浅红 = 易错与原图纠错；浅绿 = 小结与自检；左侧蓝线框 = 与其他课程的对照。
3. **“教学补全”。**课件和笔记为了省篇幅略去了一些推导、反例和条件。凡是讲义自己补上的部分，都标注“教学补全”，与原始材料区分开。
4. **练习。**正式练习共 38 道（基础 25 + 提升 10 + 跨学科 3），课件第 56–58、78 页的 4 道课堂题和第 79 页的 2 道作业题都已原样收入。答案集中放在练习之后，请先独立完成。

本章学什么、在课程里处于什么位置

第 2–4 章研究的线性表、栈、队列、串，都是“一个接一个”的线性结构：每个元素最多一个前驱、一个后继。本章开始把这个限制放宽：

- **数组**：元素仍然排得整整齐齐，但一个元素可以同时属于“一行”和“一列”，于是有了多个前驱和后继。核心问题是：计算机内存只有一条线，多维的数组怎样排进去？给出下标，怎样立刻算出地址？
- **矩阵的压缩存储**：很多矩阵有大量重复元素或零元素，全部存下来很浪费。核心问题是：只存一部分，怎样仍然能由 (i, j) 找到元素？
- **广义表**：元素本身还可以是一张表，形成“表套表”。课件只讲概念：表头、表尾、长度、深度。

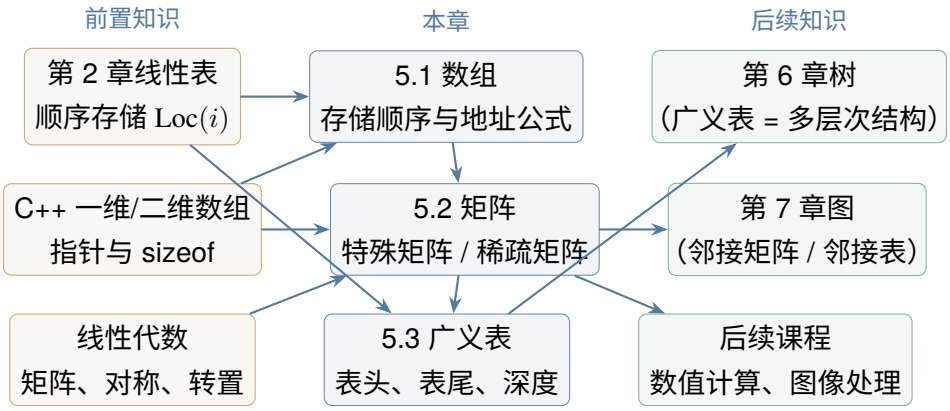
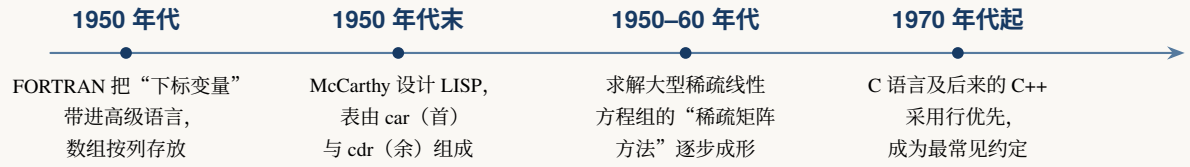


图 0.1　本章的前置知识与后续知识（箭头表示“为……做准备”）

学科小史：这些知识是怎样出现的?

学科小史

早期计算机的程序员面对的第一个问题很朴素：科学计算中到处是向量和矩阵，可内存只是一长串编号连续的存储单元。**怎样把“表格”放进“一条线”里，并且按下标快速取到元素?** 这正是本章地址公式要回答的问题。

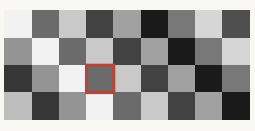


- **数组与存储顺序。**FORTRAN 规定多维数组按列存放（第一个下标变化最快），C、C++、Pascal 按行存放。两种约定并存至今，所以课件要同时讲“行序为主序”和“列序为主序”。
- **稀疏矩阵。**电网、结构力学、经济模型中的方程组动辄成千上万个未知数，但每个方程只涉及少数几个未知数，系数矩阵绝大部分是 0。只存非零元、只算非零元的“稀疏矩阵方法”由此发展起来；经济学家 H. Markowitz 因包括稀疏矩阵方法在内的工作获得 1989 年冯·诺依曼理论奖。
- **广义表。**J. McCarthy 在 1950 年代末设计 LISP 语言，以“表”为基本数据，表中元素又可以是表；取“第一个元素”和“其余部分”的两个基本操作 car / cdr，正对应本章的表头 head 与表尾 tail。

核验说明：以上只写经可靠资料确认的大致年代，不对“谁第一个提出”下结论。

从课本到现实：这一章到底有什么用?

- **图像就是二维数组。**一张 480×640 的灰度照片，在 C++ 程序里就是 `unsigned char img[480][640]`，按行存放。课件第 20 页提到的 OpenCV，其图像矩阵也按行连续存放，找像素就是用本章的行优先公式。
- **MATLAB 按列存。**控制类课程常用的 MATLAB 采用列优先（与 FORTRAN 相同），`A(:)` 把矩阵一列一列拉直。会两种公式，换语言时就不会出错。
- **稀疏矩阵无处不在。**电路的节点方程、有限元分析、社交网络的“关注”关系、推荐系统的“用户—商品”评分表，都是巨大而几乎全为 0 的矩阵。
- **广义表的影子。**文件夹里套文件夹、网页里套网页元素、JSON 里数组套数组，都是“表中有表”的多层次结构。



一张 4×9 的“小图片”：每格一个灰度值（0–255）。
按行存放时，像素 (2,3) 前面有 2 整行 + 3 个像素，
偏移 $= 2 \times 9 + 3 = 21$ 个字节——这正是第 1.4 节的公式。

图 0.2　图像 = 二维数组

原始资料重编码与纠错

材料清单与考试范围

来源	内容	本讲义处理
课件 1–15 页	数组概念、行/列序、二维与 n 维地址公式；Array 类与 vector/array 比较	公式全讲；Array 类、容器比较课件注明“不讲，了解”
课件 16–30 页	矩阵概念、Matrix 类、对称/三角/三对角矩阵压缩与类模板	全讲（含文件夹中的 3 个 C++ 程序）
课件 31–55 页	稀疏矩阵：概念、原则、基本操作、三元组顺序表、简单转置、相加相乘、带行指针的链表；十字链表	全讲；十字链表“不讲”
课件 56–58、78 页	4 道课堂练习	收入正式练习（基础单选 2、5、8，基础解答 2）
课件 60–66 页	广义表概念、长度、深度、性质、ADT、基本操作	全讲
课件 67–77 页	引用数法广义表存储结构与求深度程序	课件标“不讲”；只保留深度的递归思想
课件 79 页	作业 5.1、5.2	收入正式练习（基础解答 1、2）
课堂笔记 9 页	数组 ADT、一维/二维/三维/n 维地址、对称/三角/对角/稀疏矩阵、广义表	逐条吸收，错误处见下方纠错

原图纠错

下面这些问题都不影响你对知识的理解，但如果照抄进答卷就会丢分，所以逐条列出。

原图纠错 1：一维数组例题的地址范围（笔记第 3 页）

原始写法： `a[4]：2016~2020`。 **问题：** 每个 int 占 4 字节，$a[4]$ 占 2016、2017、2018、2019 这 4 个字节，2020 已经是数组之外的下一个单元。 **规范写法：** `a[4]：2016~2019`。
影响： 只影响这一行，公式 $\mathrm{Loc}(i)=a+iL$ 本身正确。

原图纠错 2：三维和 n 维地址公式漏了“$\times L$”（笔记第 4 页）

原始写法： $\mathrm{Loc}(i_1,i_2,i_3)=a+i_1m_2m_3+i_2m_3+i_3$，以及 $\mathrm{Loc}(i_1,\ldots,i_n)=a+\sum_{j=1}^{n-1}i_j\prod_{k=j+1}^{n}m_k+i_n$。
问题： 括号里算出的是“前面有多少个元素”，要换成地址还得乘每个元素占的单元数 L。只有 $L=1$ 时原式才碰巧成立。
规范写法： $\mathrm{Loc}(i_1,i_2,i_3)=a+(i_1m_2m_3+i_2m_3+i_3)\,L$；$n$ 维同理整体乘 L（见第 1.4 节）。

原图纠错 3：数组 ADT 的数据关系下标（笔记第 2 页）

原始写法： $0 \le j_i \le b_k - 2$。　**问题：**这里限制的是第 i 维的下标 j_i，它的上界应当用第 i 维的长度 b_i。　**规范写法：** $0 \le j_i \le b_i - 2$（第 i 维下标最大到 $b_i - 1$，有后继的元素最大到 $b_i - 2$）。

原图纠错 4：三对角矩阵的下标公式与所画存储图不一致（笔记第 6 页）

原始写法：存储图为 $\boxed{0}\boxed{a_{00}}\boxed{a_{01}}\boxed{a_{10}}\cdots\boxed{a_{66}}$，标 $k = 0, 1, 2, \ldots, 19$；随后写“第 1 行 $a_{00}, a_{01} \to 0, 1$”，结论 $k = 2i + j$。

问题：按图，$k = 0$ 被常数 0 占用，a_{00} 在 $k = 1$，a_{66} 在 $k = 19$；而 $2i + j$ 给出 $a_{00} \to 0$、$a_{66} \to 18$，两者差 1。

规范写法： i, j 从 0 开始时，若 `sa[0]` 存常数，则 $k = 2i + j + 1$；若不存常数、从 `sa[0]` 直接存 a_{00}，则 $k = 2i + j$。换成 i, j 从 1 开始，前者就是课件第 28 页的 $k = 2(i - 1) + j$。详见第 2.5 节。

原图纠错 5：稀疏矩阵三元组例题的“总体信息”（笔记第 7 页）

原始写法：笔记中的矩阵是 6 行 5 列、共 7 个非零元（12, 9, −3, 14, 24, 18, 15），但总体信息行写成 `6 6 8`。

问题：总体信息应为（行数，列数，非零元个数）=（6, 5, 7）。`6 6 8` 可能来自另一本教材里一个 6 行 7 列、8 个非零元的同类例题（原始资料无法确认）。

规范写法：`(6, 5, 7)`，其后 7 个三元组笔记写得正确。

原图纠错 6：两处措辞（笔记第 4 页）

(a) “常规存储：一个二元数组”应为“一个**二维**数组”。**(b)** 笔记把稀疏矩阵列为第 4 种“特殊矩阵”。课件第 21 页的“特殊矩阵”指非零元**分布有规律**的矩阵（对称、三角、三对角），能用公式映射；稀疏矩阵的非零元**分布没有规律**，只能逐个记录位置。两者压缩方法完全不同，考试中按课件区分。

（课件第 64 页广义表 ADT 中的一处标签问题，见第 3.5 节“原图纠错 7”。）

☞ 教学补全：笔记中两处需要补充说明的地方

- “$\delta \le 0.05$ **为稀疏矩阵**”。这是常用的经验标准，不是严格定义。课件第 31 页的定义是“非零元素个数远远少于矩阵元素个数”。课件里的小例子（如 6×7 矩阵有 8 个非零元，$\delta \approx 0.19$）只是为了演示方法，并不满足 0.05。
- **“以行序为主序（低下标优先）”**。笔记的括号说法来自部分教材，含义容易误解。最稳妥的记法是：**行优先 = 最右边的下标变化最快；列优先 = 最左边的下标变化最快。**
- **三角矩阵的三套公式**。课件（`sa[0]` 存常数）、笔记（$B[1..]$ 存元素，常数放最后）和对称矩阵（`sa[0]` 起存元素）用的是三种不同的下标约定，公式看起来不同但都对。第 2.4 节有一张总对照表。

知识点总览

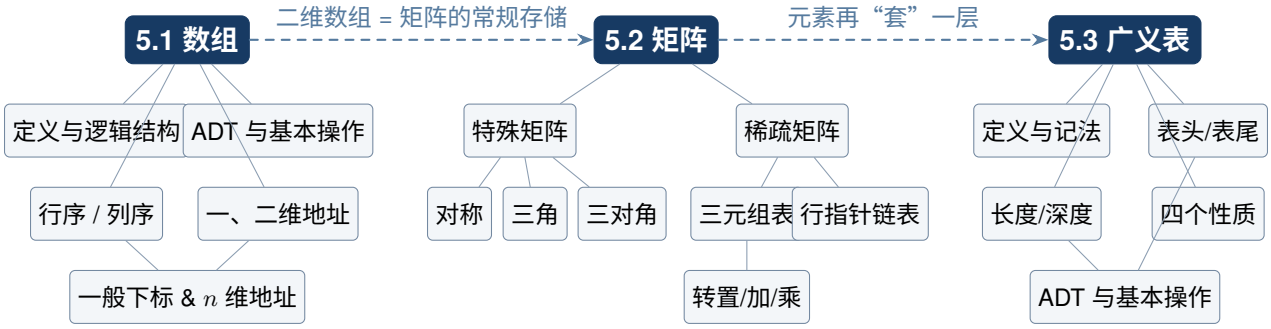


图 0.3　本章知识点总览

第 1 章　数组：把“表格”排进一条线

本章回答三个问题： (1) 数组和线性表是什么关系？(2) 多维数组在内存里按什么顺序排？(3) 给出下标，怎样一步算出存储地址？

知识路线： 数组的定义 → 逻辑结构 → ADT 与基本操作 → 行序 / 列序 → 一维 → 二维 → 一般下标 → 三维 / n 维地址。

1.1　数组的基本概念

它解决什么问题

第 2 章的线性表 $(a_1, a_2, \ldots, a_n)$ 里，每个元素只有“前一个”和“后一个”。可是成绩单（学生 × 科目）、图像（行 × 列）、矩阵这类数据，天然有两个方向：同一个元素既在某一行里，又在某一列里。描述这种“横竖都整齐”的数据，就需要数组。

❑ 数组（课件第 3 页 / 笔记第 1 页）

数组是由**相同类型**的一组数据元素组成的一个**有限序列**（笔记：按一定格式排列起来的、具有相同类型的数据元素的集合）。数组元素通常也称为数组元素；每个元素有一个序号，称为**下标**，可以通过下标访问元素。

定义里有三个关键词，缺一不可：

- **相同类型**：所以每个元素占的存储单元数 L 都一样——这是后面所有地址公式的前提；
- **有限**：元素个数在定义时就确定了；
- **按下标访问**：下标就是元素的“坐标”。

正例： `int score[40];`（40 个同类型的 int）。　**反例：** 一个结构体 `{string name; int age;}` 的两个成员不是一个数组——类型不同，也不能用下标访问。

一维、二维、多维

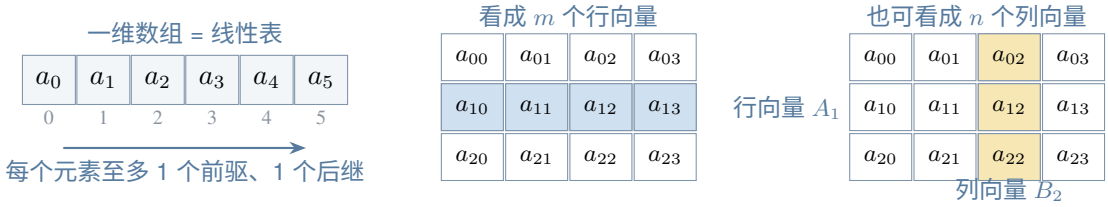


图 1.1　一维数组就是线性表；二维数组既可按行、也可按列看成“由向量组成的向量”

- **一维数组**：线性表中的元素是非结构的简单元素（笔记），它就是第 2 章顺序存储的线性表。
- **二维数组**：一维数组的元素本身又是一维数组——即“数组的数组”。设 A 有 m 行 n 列（课件第 5 页、笔记第 2 页）：

$$A=[A_0,A_1,\ldots,A_{m-1}],\quad A_i=(a_{i0},a_{i1},\ldots,a_{i,n-1}),\ 0\le i\le m-1;$$

$$A=[B_0,B_1,\ldots,B_{n-1}],\quad B_j=(a_{0j},a_{1j},\ldots,a_{m-1,j}),\ 0\le j\le n-1.$$

  二维数组又称**矩阵**，任一元素有两个下标：行号和列号。
- **多维数组**：三维数组可视为以二维数组为元素的向量，四维数组以三维数组为元素……依此类推（课件第 6 页）。

逻辑结构：二维数组到底是不是线性结构？

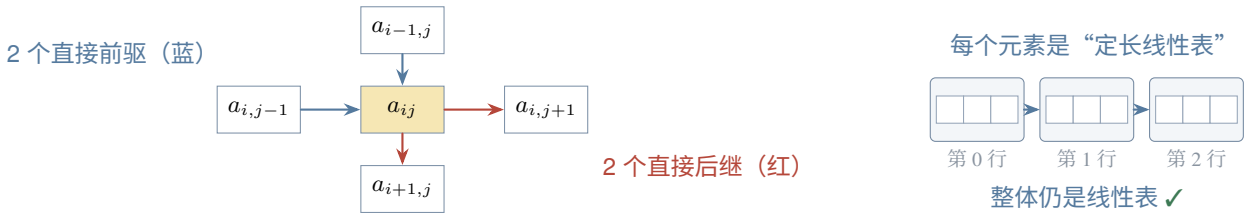


图 1.2　左：逐个元素看，a_{ij} 同时在行表和列表中——非线性；右：把“一整行”当作一个元素——线性

两种说法都出现在笔记里，它们并不矛盾，而是**看问题的粒度不同**：

- **以单个元素为单位**（课件第 5 页的结论）：a_{ij} 既属于第 i 行的行向量，又属于第 j 列的列向量，最多有两个直接前驱和两个直接后继（边界除外），所以二维数组是**典型的非线性结构**；三维数组每个元素最多有 3 个前驱、3 个后继，多维数组同样是非线性结构。
- **以“一整行”为单位**：线性表的每个元素又是一个定长的线性表，于是二维数组可以看成线性表的推广。

考试问“二维数组的逻辑结构”时，按课件答“非线性结构”，必要时补一句“也可视为线性表的推广”。

✘ 易错点

误区：“二维数组在内存里是连续存放的一条线，所以它是线性结构。”
错在哪：混淆了**逻辑结构**和**存储结构**。“存成一条线”说的是存储结构（顺序存储）；“一个元素有几个前驱后继”说的才是逻辑结构。
正确想法：逻辑上非线性，存储上用顺序存储把它“压平”成一条线——这正是 1.3 节要研究的事。

在 C++ 中声明数组

笔记给出的声明格式是 数据类型变量名称 [长度]、数据类型变量名称 [行数][列数]。笔记第 2 页还写了“int array1[n]、array1 array2[m] 等价于二维数组”，在 C++ 里完整的写法要用 typedef：

C++　02_typedef.cpp：数组的数组

```cpp
// 02_typedef.cpp  “数组的数组”：用 typedef 声明二维数组
#include <iostream>
using namespace std;

const int N = 4;                    // 每行 4 个元素（长度必须是常量表达式）
typedef int Row[N];                 // Row 是“含 4 个 int 的一维数组”类型

int main()
{
    Row a[3];                       // a 是“含 3 个 Row 的一维数组”，等价于 int a[3][4]
    int b[3][4];
    cout << sizeof(a) << " " << sizeof(b) << endl;     // 48 48
    cout << sizeof(a[0]) << endl;                      // 一行 = 16 字节
    return 0;
}
```

运行输出（g++ 实测）

```
48 48
16
```

这段代码说明了什么：Row a[3] 和 int b[3][4] 占用完全相同的 48 字节，a[0] 是一整行（16 字节）。C++ 的二维数组本来就是“元素为一维数组的一维数组”。

✘ 易错点：数组长度必须是常量

int n; cin >> n; int a[n]; 在标准 C++ 中**不合法**（这叫变长数组，只是某些编译器的扩展），考试代码里不要这样写。长度要用 const int 常量或字面值；运行时才知道大小时，用 new int[n]（课件中的类模板都是这样分配空间的）。

1.2　数组的特点、基本操作与抽象数据类型

❑ 数组的特点与基本操作（笔记第 2–3 页）

特点：结构固定——数组一旦定义，**维数**和**维界**（每一维的下标范围）就不再改变。
基本操作只有四种：①结构初始化；②销毁；③取元素；④修改元素的值。

为什么 / 直观理解

为什么没有“插入”和“删除”？线性表可以在第 i 个位置插入，表长加 1；可二维数组若在中间插入一个元素，这一行就比别的行多出一个，表格“不整齐”了，下标与位置的对应关系也被破坏。所以数组只允许读和改已有位置上的值，不改变结构。正是“结构固定”，才让“给下标、直接算地址”成为可能，也就让数组天然适合**顺序存储**。

❑ 数组的抽象数据类型（笔记第 2 页，已按纠错 3 修正）

ADT Array {
数据对象： $j_i = 0, \ldots, b_i - 1,\ i = 1, 2, \ldots, n;\quad D = \{a_{j_1 j_2 \cdots j_n} \mid a_{j_1 j_2 \cdots j_n} \in \text{ElemSet}\}$
数据关系： $R = \{R_1, R_2, \ldots, R_n\}$，其中
$R_i = \{\langle a_{j_1 \cdots j_i \cdots j_n},\ a_{j_1 \cdots j_i + 1 \cdots j_n} \rangle \mid 0 \le j_k \le b_k - 1,\ 1 \le k \le n \text{ 且 } k \ne i,\ 0 \le j_i \le b_i - 2\}$
基本操作：初始化、销毁、取元素、修改元素值
}

逐个符号读懂它：

- n 是维数；b_i 是第 i 维的长度，所以第 i 维下标 j_i 取 $0 \sim b_i - 1$。
- R_i 是“沿第 i 维走一步”的关系：其他下标都不动，只把 j_i 加 1。
- $0 \le j_i \le b_i - 2$：只有不在第 i 维最后一个位置的元素，才有沿该维的后继。

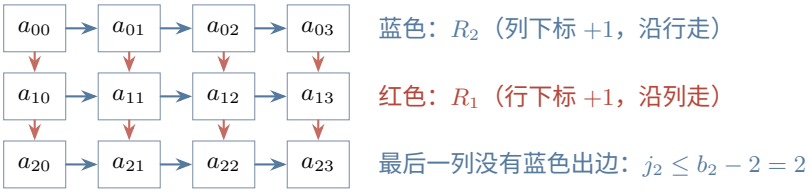


图 1.3　二维数组（$b_1 = 3,\ b_2 = 4$）的数据关系 R_1, R_2

★ 生活中的它：电影院的座位

生活场景：电影票上印着“5 排 8 座”。
对应关系：放映厅 = 二维数组；排号 = 行下标；座号 = 列下标；每个座位“同类型”（都是一个座位）；座位数在建好后固定不变 = **结构固定**；观众可以换人（修改元素值），但不能临时在两个座位中间“插入”一个座位。
小判断：5 排 8 座的“前后邻居”是 4 排 8 座和 6 排 8 座，“左右邻居”是 5 排 7 座和 5 排 9 座——正好 2 个前驱、2 个后继，这就是二维数组的非线性。
边界说明：真实影厅常有过道、每排座位数不同，严格说不是整齐的矩形数组。

➢ 对照：数组 ↔ 线性表的顺序表（第 2 章）（推广型 / 方法互补型）

一句话定位：一维数组就是顺序存储的线性表；多维数组把“一个方向”推广成“多个方向”，代价是结构固定、不再插入删除。

比较维度	顺序表（第 2 章）	数组（本章）
逻辑结构	线性：1 个前驱、1 个后继	一维线性；二维及以上非线性
结构是否可变	表长可变，可插入、删除	维数和维界固定
基本操作	插入、删除、查找、取值……	只有初始化、销毁、取元素、改值
地址公式	$\mathrm{Loc}(a_i)=\mathrm{Loc}(a_1)+(i-1)L$	多维：前面有多少个元素 $\times L$

相同之处：都用顺序存储，都能按下标随机存取，存取时间与位置无关。

不同之处及根源：数组多了“维”。只要允许插入，就会破坏“每行一样长”，地址公式随之失效——这就是数组放弃插入删除的根本原因。

顺序表中的结论	迁移到数组	理由
按下标随机存取，$O(1)$	✓	地址仍是下标的线性函数
地址 = 基址 + 前面元素个数 $\times L$	✓	只是“前面的元素个数”要按维数去数
可以在中间插入一个元素	✗	会破坏结构固定，二维中某一行变长
第 2 章公式下标从 1 开始	△	C++ 数组从 0 开始，公式中的 -1 要去掉

了解即可（课件第 13–15 页注明“不讲”）

课件给出了一个通用的 `Array` 类模板（用 `base` 存元素、`bounds` 存各维长度、`constants` 存映像函数常量），并比较了 C++ 内置数组、`vector` 与 `array`：内置数组和 `array` 大小在编译时固定；`vector` 可在运行时改变大小、自动管理内存。考试写代码时，直接用 C++ 内置数组即可（课件第 14 页原话：“使用中直接使用 C++ 数组即可”）。

1.3　数组元素的存储顺序：行序与列序

内存是一条线，二维数组要“压平”才能放进去，压平的方式有两种。

❑ 两种存储顺序（课件第 7 页）

以行序为主序（按行排列）：先存第一行的所有元素，再存第二行，依次类推。

例：3×3 数组存为 $a_{00},a_{01},a_{02},a_{10},a_{11},a_{12},a_{20},a_{21},a_{22}$。

以列序为主序（按列排列）：先存第一列的所有元素，再存第二列，依次类推。

例：存为 $a_{00},a_{10},a_{20},a_{01},a_{11},a_{21},a_{02},a_{12},a_{22}$。



图 1.4　同一个 3×3 数组的两种“压平”方式（下方为内存中的顺序与序号）

读图：红色箭头是存储时走过的路线；下方一条格子就是内存，灰色小字是“前面有几个元素”。同一个 a_{12}，行序下排第 5 号，列序下排第 7 号——**位置取决于约定**。

为什么 / 直观理解：一句话记住

行优先：最右边的下标变化最快（$a_{00} \to a_{01} \to a_{02}$，先变列号）；**列优先：最左边的下标变化最快**（$a_{00} \to a_{10} \to a_{20}$，先变行号）。这条规则对三维、$n$ 维同样成立。

★ 生活中的它：横排书和竖排古籍

生活场景：现代横排书从左到右读完一行再换行；传统竖排古籍从上到下读完一列再换下一列。

对应关系：一页书 = 二维数组；读书顺序 = 存储顺序；横排 = 行序为主序，竖排 = 列序为主序。

小判断：每行 30 字的横排书，第 3 行第 5 个字（都从 0 数）是本页第 $3 \times 30 + 5 = 95$ 号字。

边界说明：古籍竖排是从右往左换列，与列优先“从左往右换列”的方向不同，只是“先走完一列”这一点相同。

1.4　元素存储位置的计算

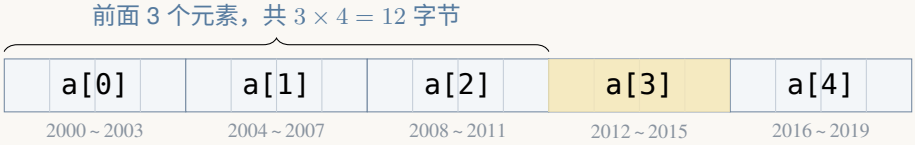
核心思想：地址 = 基地址 +（前面有多少个元素）$\times L$

数组元素类型相同，每个占 L 个存储单元，又是一个挨一个连续存放的。所以只要数出“**排在它前面的元素有几个**”，乘以 L，再加上第一个元素的地址（**基地址**）就行了。本节所有公式都只是用不同方式去“数前面有几个”。

(1) 一维数组

例 1.1（笔记第 3 页）　一维数组的地址

`int a[5];` 每个元素占 4 字节，设 `a[0]` 存储在 2000 单元，`a[3]` 的地址是多少?



已知： $\mathrm{Loc}(0)=a=2000$，$L=4$。　**求：** $\mathrm{Loc}(3)$。
思路： `a[3]` 前面有 `a[0]`、`a[1]`、`a[2]` 三个元素。
计算： $\mathrm{Loc}(3)=a+3L=2000+12=2012$，它占 2012–2015 四个字节。
检查： `a[4]` 占 2016–2019（不是 2020，见纠错 1），整个数组占 2000–2019 共 20 字节 $=5\times4$。
✓

Σ 一维数组地址公式（笔记第 3 页）

$$\mathrm{Loc}(i)=\begin{cases}\mathrm{Loc}(0)=a, & i=0\\ \mathrm{Loc}(i-1)+L=a+i\cdot L, & i>0\end{cases}\qquad \boxed{\mathrm{Loc}(i)=a+i\cdot L}$$

a：基地址（`a[0]` 的地址）；i：下标（C++ 中从 0 开始），恰好等于前面元素的个数；L：每个元素占的单元数。

公式从哪来： 递推式 $\mathrm{Loc}(i)=\mathrm{Loc}(i-1)+L$ 说“下一个元素紧挨着上一个”。从 $\mathrm{Loc}(0)=a$ 出发连续用 i 次，每次加 L，就得到 $a+iL$。

(2) 二维数组（C++ 下标从 0 开始）

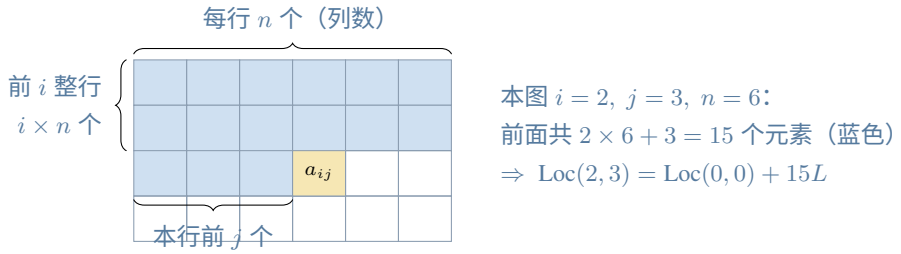


图 1.5　行序为主序时，a_{ij} 前面的元素 = 前 i 整行 + 本行前 j 个

Σ 二维数组地址公式（课件第 8 页 / 笔记第 4 页）

设二维数组 `a[m][n]`（m 行 n 列，下标从 0 开始），每个元素占 L 个单元：

行序为主序：$\boxed{\mathrm{Loc}(i,j)=\mathrm{Loc}(0,0)+(n\cdot i+j)\cdot L}$

列序为主序：$\boxed{\mathrm{Loc}(i,j)=\mathrm{Loc}(0,0)+(m\cdot j+i)\cdot L}$

$\mathrm{Loc}(0,0)$ 称为**基地址**或**基址**；$n\cdot i+j$ 正是 a_{ij} 前面所有元素的个数（笔记红字）。课件写作 $(b_2\times i+j)\times L$，b_2 就是第 2 维长度（列数）n。

推导：行序为主序

要证： 行序存放时，a_{ij} 前面恰有 $n\cdot i+j$ 个元素。
依据： 行序为主序的定义——第 $0,1,\ldots,i-1$ 行全部存完，才轮到第 i 行；第 i 行内部按列号 $0,1,\ldots$ 顺序存放。
第一步： 第 $0\sim i-1$ 行共 i 行，每行 n 个，共 $i\cdot n$ 个元素，都排在 a_{ij} 前面。
第二步： 在第 i 行内，排在 a_{ij} 前面的是 $a_{i0},\ldots,a_{i,j-1}$，共 j 个。

第三步：其余元素（第 i 行中 j 之后的元素和第 $i+1$ 行以后）都在 a_{ij} 之后。
结论：前面共 $ni+j$ 个，故 $\mathrm{Loc}(i,j)=\mathrm{Loc}(0,0)+(ni+j)L$。 □
列序同理——但不能只写“同理”：列序时先存完第 $0\sim j-1$ 列，每列有 m 个（**行数**），再加上第 j 列中前面的 i 个，所以是 $m\cdot j+i$。注意乘的是**行数** m，而行序乘的是**列数** n。

✘ 易错点

最常见的错：行序公式里乘成了行数。记法：行序时“跳过一整行”要跳过的是一行里的元素个数，也就是**列数**。
第二常见的错：忘了乘 L，或者把基地址当成 0 忘了加。

用 C++ 亲眼验证：下面的程序用指针取出每个元素的真实地址，与公式比较。

C++　01_address.cpp：验证行优先地址公式

```cpp
// 01_address.cpp  用 C++ 指针验证“行优先”地址公式
#include <iostream>
using namespace std;

int main()
{
    const int M = 3, N = 4;
    int a[M][N];                              // C++ 二维数组：按行序为主序连续存放
    char *base = (char *)&a[0][0];            // 以字节为单位的基地址
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            long real = (char *)&a[i][j] - base;          // 实际偏移（字节）
            long calc = (long)(N * i + j) * sizeof(int);  // 公式 (n*i+j)*L
            cout << real << (real == calc ? " " : "!") ;  // 不相等就打印 !
        }
        cout << endl;
    }
    cout << "sizeof(a) = " << sizeof(a) << endl;          // 3*4*4 = 48

    int b[2][3][4];                           // 三维：b[i1][i2][i3]
    long off = (char *)&b[1][2][3] - (char *)&b[0][0][0];
    cout << "b[1][2][3] offset = " << off
         << ", formula = " << (1 * 3 * 4 + 2 * 4 + 3) * sizeof(int) << endl;
    return 0;
}
```

运行输出（g++ 实测）

```
0 4 8 12
16 20 24 28
32 36 40 44
sizeof(a) = 48
b[1][2][3] offset = 92, formula = 92
```

读输出：每行 4 个 int 正好 16 字节，第 1 行从 16 开始，没有一个“!”——C++ 的二维数组确实按行序为主序存放，公式与实际地址完全一致。第 9–15 行的双重循环就是“逐个元素对答案”，三维的验证留到下文。

(3) 一般下标的二维数组 a[c_1:d_1, c_2:d_2]

课件第 9 页考虑下标不从 0 开始的情形：第一维下标从 c_1 到 d_1，第二维从 c_2 到 d_2。课堂练习（课件第 56 页）就是这种题。

Σ 一般下标二维数组（课件第 9 页）

设每个元素占 L 个单元，$\mathrm{Loc}(c_1, c_2)$ 是第一个元素 $a_{c_1c_2}$ 的地址：

$$\text{行序：}\quad \mathrm{Loc}(i,j) = \mathrm{Loc}(c_1,c_2) + [(d_2-c_2+1)(i-c_1)+(j-c_2)]\,L$$

$$\text{列序：}\quad \mathrm{Loc}(i,j) = \mathrm{Loc}(c_1,c_2) + [(d_1-c_1+1)(j-c_2)+(i-c_1)]\,L$$

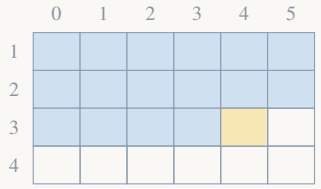
推导：把下标“平移”回从 0 开始

令 $i' = i - c_1$，$j' = j - c_2$，则 i', j' 都从 0 开始；每行元素个数（列数）为 $n = d_2 - c_2 + 1$，每列元素个数（行数）为 $m = d_1 - c_1 + 1$。代入 0 起始的公式 Loc = 基址 + $(ni' + j')L$ 即得行序公式；列序同理得 $(mj' + i')L$。

关键提醒：个数是 $d - c + 1$，不是 $d - c$。例如下标 $2 \sim 6$ 共有 $6 - 2 + 1 = 5$ 个。

例 1.2　一般下标数组

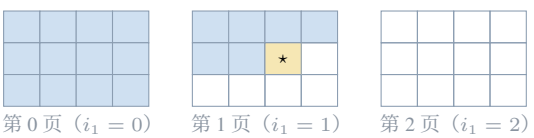
数组 A[1:4, 0:5]，每个元素占 2 个单元，$\mathrm{Loc}(1,0) = 200$。分别按行序、列序求 $a_{3,4}$ 的地址。



行：$1 \sim 4$，共 4 行；列：$0 \sim 5$，共 6 列
行序：前面 2 整行 $+4$ 个 $= 16$ 个（蓝色）

行序：$m = 4,\ n = 6$；$\mathrm{Loc}(3,4) = 200 + [6 \times (3-1) + (4-0)] \times 2 = 200 + 16 \times 2 = 232$。
列序：$\mathrm{Loc}(3,4) = 200 + [4 \times (4-0) + (3-1)] \times 2 = 200 + 18 \times 2 = 236$。
检查：最后一个元素 $a_{4,5}$ 行序偏移 $6 \times 3 + 5 = 23$，$23 + 1 = 24 = 4 \times 6$ 个元素，合理。
本题想说明：先把下标“平移到从 0 开始”，再套 0 起始公式；行列数一定用 $d - c + 1$。

(4) 三维数组与 n 维数组



目标 $a[1][1][2]$（★）：
前面 1 整页 $= 12$ 个
+ 本页 1 整行 $= 4$ 个
+ 本行 2 个
$= 18$ 个（蓝色）

图 1.6　三维数组按“页—行—列”分层计数（行序为主序）

三维数组 a[m_1][m_2][m_3] 可看成 m_1 “页”，每页是 $m_2 \times m_3$ 的二维数组（上图 $m_1 = 3, m_2 = 3, m_3 = 4$）。求 $a[i_1][i_2][i_3]$ 前面的元素：① 前面 i_1 整页，$i_1 m_2 m_3$ 个；② 本页前面 i_2 整行，$i_2 m_3$ 个；③ 本行前面 i_3 个。

Σ 三维数组（笔记第 4 页，已补 L）

`a[`m_1`][`m_2`][`m_3`]`，各维元素个数 m_1, m_2, m_3，行序为主序，基地址 a：

$$\boxed{\mathrm{Loc}(i_1, i_2, i_3) = a + (i_1 \cdot m_2 \cdot m_3 + i_2 \cdot m_3 + i_3) \cdot L}$$

Σ n 维数组（课件第 10 页）

设各维长度为 $b_1, b_2, \ldots, b_n$，下标从 0 开始，行序为主序：

$$\begin{aligned}\mathrm{Loc}(j_1, j_2, \ldots, j_n) &= \mathrm{Loc}(0, 0, \ldots, 0) + \big[\, j_1 b_2 \cdots b_n + j_2 b_3 \cdots b_n + \cdots + j_{n-1} b_n + j_n \,\big] L \\ &= \mathrm{Loc}(0, 0, \ldots, 0) + \sum_{i=1}^{n} c_i \, j_i\end{aligned}$$

其中 $\boxed{c_n = L, \quad c_{i-1} = b_i \times c_i \ (1 < i \le n)}$。**数组元素的存储位置是其下标的线性函数。**

推导：系数 c_i 的含义

c_i 是“第 i 维下标加 1 时，地址增加多少”。

最后一维： j_n 加 1，就是同一行中的下一个元素，地址加 L，所以 $c_n = L$。

倒数第二维： j_{n-1} 加 1，要跳过一整“行”，这一行有 b_n 个元素，地址加 $b_n L = b_n c_n$，所以 $c_{n-1} = b_n c_n$。

一般地： j_{i-1} 加 1，要跳过一整块“第 i 维到第 n 维”组成的子数组，它含 $b_i b_{i+1} \cdots b_n$ 个元素，地址加 $b_i \cdot (b_{i+1} \cdots b_n L) = b_i c_i$。这就是递推式 $c_{i-1} = b_i c_i$。

展开得 $c_i = L \cdot b_{i+1} b_{i+2} \cdots b_n$，与方括号里的写法完全一致。 □

为什么重要： 地址是下标的线性函数，计算量与数组多大无关——这就是数组能**随机存取**的原因。

课件第 11–12 页还给出了一般下标 `a[`c_1`:`d_1`,...,`c_n`:`d_n`]` 的公式：按行存放时

$$\begin{aligned}\mathrm{Loc}(j_1 \ldots j_n) = \mathrm{Loc}(c_1, \ldots, c_n) + \big[&(j_1 - c_1)(d_2 - c_2 + 1) \cdots (d_n - c_n + 1) \\ &+ \cdots + (j_{n-1} - c_{n-1})(d_n - c_n + 1) + (j_n - c_n)\big] L,\end{aligned}$$

按列存放时，把乘积的方向反过来（最左边的下标变化最快）。思路完全一样：平移下标，再数“前面有几个”。

例 1.3　三维数组

`int b[2][3][4];`（int 占 4 字节），求 `b[1][2][3]` 相对 `b[0][0][0]` 的偏移。

前面的元素： $1 \times (3 \times 4) + 2 \times 4 + 3 = 12 + 8 + 3 = 23$ 个；　**偏移：** $23 \times 4 = 92$ 字节。

用 c_i 再算一次： $c_3 = 4$，$c_2 = b_3 c_3 = 16$，$c_1 = b_2 c_2 = 48$；偏移 $= 48 \times 1 + 16 \times 2 + 4 \times 3 = 92$。

检查： 与上面 `01_address.cpp` 的实测输出“offset = 92”一致。✓

★ 生活中的它：时、分、秒的换算

生活场景：秒表显示 2:03:04，问一共过了多少秒?

对应关系：把“时、分、秒”看成三维下标 (i_1, i_2, i_3)，各维长度 $(24, 60, 60)$；“过了多少秒” = 前面有多少个元素。

小估算：$2 \times 60 \times 60 + 3 \times 60 + 4 = 7384$ 秒——与三维行序公式 $i_1 m_2 m_3 + i_2 m_3 + i_3$ 一模一样（$L = 1$）。

边界说明：时间可以无限往前走，而数组每一维有上界；类比的只是“逐级进位计数”这一点。

➢ 对照：行优先地址公式 ↔ 进位计数制（类比同构型）

一句话定位：行优先下的“前面元素个数”就是把下标 $(j_1, \ldots, j_n)$ 看成一个**混合进制数**，各位的“进制”是 $b_1, \ldots, b_n$。

	十进制数 / 时分秒	行优先数组下标
各位	百位、十位、个位 / 时、分、秒	$j_1, j_2, \ldots, j_n$
每位的权	$100, 10, 1$ / $3600, 60, 1$	$b_2 \cdots b_n$，$\ldots$，b_n，1（再乘 L）
“最低位”变化最快	个位 / 秒	最右边的下标 j_n

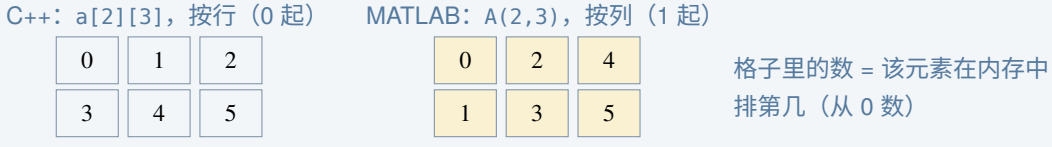
不同之处及根源：普通进制每位权相同（都是 10），数组每一维长度可以不同，所以是“混合进制”；列优先相当于把最左边的下标当作“个位”。

进制中的结论	迁移	理由
数值 = ∑(位值 × 权)	✓	即 $\sum c_i j_i$
每一位取值 0 ∼ (进制 − 1)	✓	即 $0 \le j_i \le b_i - 1$
最高位（j_1）的取值范围影响权	✗	b_1 根本不出现在公式中：第一维多长不影响任何元素的偏移
下标不从 0 开始也能直接代	✗	必须先平移成 $j_i - c_i$

同一个小算例：十进制“345” $= 3 \times 100 + 4 \times 10 + 5$；数组 `a[10][10][10]` 中 $a[3][4][5]$ 前面恰有 345 个元素。

➢ 对照：C++ 的行优先 ↔ MATLAB / FORTRAN 的列优先（思想重现型）

一句话定位：同一套“数前面有几个”的思想，换一种存储约定、换一种下标起点。控制专业后续常用 MATLAB，这个对照很实用。



C++ 中的结论	迁移到 MATLAB	理由或反例
地址是下标的线性函数	✓	两种约定都是线性函数
`a[i][j]` 与 `a[i][j+1]` 在内存中相邻	✗	列优先时相邻的是 `A(i,j)` 与 `A(i+1,j)`
偏移 $= ni + j$	△	改为 $m(j-1)+(i-1)$：先换列序，再处理 1 起始

(5) 课件中的 Matrix 类：1 起始下标映射到 0 起始数组

课件第 18–19 页的矩阵类把 $m \times n$ 个元素按行序存在一维数组 `elems` 中，但对外使用 1 起始的行号、列号。核心只有一行：

C++　03_matrix.cpp：重载 () 运算符取元素

```cpp
template <class ElemType>
class Matrix {
protected:
    ElemType *elems;              // 按行序为主序存放全部 rows*cols 个元素
    int rows, cols;
public:
    Matrix(int rs, int cs) : rows(rs), cols(cs) { elems = new ElemType[rs * cs]; }
    ~Matrix() { delete []elems; }
    ElemType &operator()(int row, int col)
    {
        if (row < 1 || row > rows || col < 1 || col > cols) {
            cout << "下标出界!" << endl; exit(2);
        }
        return elems[(row - 1) * cols + col - 1];   // 前面有 row-1 整行 + col-1 个
    }
};
```

逐句解释： `operator()` 返回引用 `ElemType &`，所以 `a(2,3)=6;` 能写、`cout<<a(2,3);` 能读——这就是数组 ADT 的“取元素”和“修改元素值”。下标先检查范围；行号、列号各减 1 变成 0 起始，再套行序公式 $(\text{row}-1)\cdot\text{cols}+(\text{col}-1)$。$3\times 3$ 矩阵中 `a(2,3)` 对应 `elems[5]`。

了解即可（课件第 20 页）

C++ 本身没有“矩阵类”。大量矩阵运算可选用专门的库，如 Armadillo、Eigen；处理图像时常用 OpenCV。不同的库存储约定可能不同（例如 Eigen 默认按列存储），用之前查清楚即可。

本章知识地图与小结

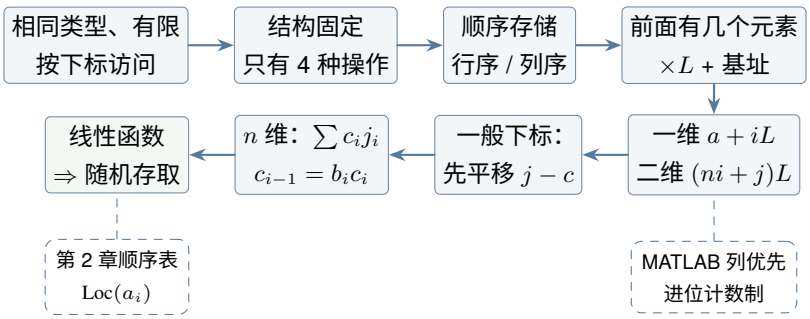


图 1.7　第 1 章知识地图（虚线：与其他课程知识的对照）

✓ 到这里，我们已经解决了什么？

数组元素类型相同、结构固定，所以可以顺序存储；按行序或列序“压平”后，任一元素的地址都等于“基址 + 前面元素个数 $\times L$”，而“前面元素个数”是下标的线性函数——给出下标就能直接算出地址。

下一步还需要解决什么？二维数组就是矩阵的“常规存储”。但如果一个 1000×1000 的矩阵是对称的，或者 99.9% 都是 0，把一百万个元素全部存下来就太浪费了。能不能只存一部分，又仍然能用 (i, j) 找到元素？这就是第 2 章的“矩阵压缩存储”。

学完这里，你应该能够回答

1. 数组为什么只有“取元素”和“改元素”，没有插入删除？
2. 二维数组的逻辑结构是线性还是非线性？为什么两种说法都能见到？
3. 行优先公式中乘的是行数还是列数？列优先呢？
4. 下标 $-3 \sim 1$ 共有几个？代入一般下标公式时应怎样处理？
5. 写出 n 维数组的系数递推式 $c_{i-1} = ?$，并说明 $c_n = L$ 的含义。
6. 为什么说“存储位置是下标的线性函数”意味着随机存取？

第 2 章　矩阵的压缩存储：只存“有用的那部分”

本章回答三个问题：(1) 有规律的矩阵（对称、三角、三对角）怎样只存一部分，并用公式由 (i, j) 找到元素？(2) 没有规律、绝大部分是 0 的稀疏矩阵怎样存？(3) 稀疏矩阵的转置、相加、相乘怎样做？

知识路线：常规存储 → 压缩原则 → 对称 → 三角 → 三对角 → 稀疏矩阵 → 三元组顺序表 → 转置 / 加 / 乘 → 带行指针的链表。

✘ 本章下标约定（务必先看）

本章矩阵元素 a_{ij} 的行号 i、列号 j **从 1 开始**（与课件第 22–28 页及文件夹中的 C++ 类一致）；压缩数组的下标从 0 还是从 1 开始，每个公式都会单独注明。

2.1　矩阵与常规存储

❑ 矩阵（课件第 17 页 / 笔记第 4 页）

由 $m \times n$ 个数 a_{ij} 排成的 m 行 n 列的数表，称为 m 行 n 列的矩阵，简称 $m \times n$ 矩阵。行数与列数都等于 n 的矩阵称为 n 阶方阵。

注意（课件）：数组可以是一维、二维乃至多维；矩阵就是二维。矩阵是数学概念，数组是计算机上的概念；矩阵的元素只能是数，数组的元素可以是字符等。

常规存储就是用一个二维数组存放矩阵（笔记）。它的特点是：① 可以对元素**随机存取**；② 矩阵运算非常简单；③ **存储密度为 1**（每个存储单元都存放一个矩阵元素）。

但是，当矩阵中有大量**值相同**的元素或大量**零元素**时，常规存储就很浪费：

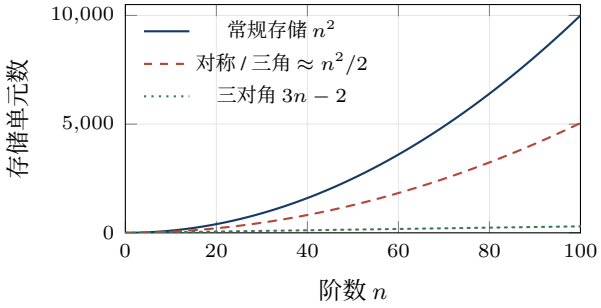


图 2.1　n 阶矩阵所需单元数：压缩后对称矩阵省一半，三对角矩阵从 n^2 降到线性

❑ 压缩存储的两条原则（笔记第 4 页 / 课件第 33 页）

① 值相同的多个元素只分配一个存储空间；　② 零元素不分配存储空间。
对稀疏矩阵，课件还要求：尽可能少存或不存零值元素；尽可能减少没有实际意义的运算；操作方便，即尽可能快地找到与下标 (i,j) 对应的元素。

压缩之后最关键的问题是：**原来的 (i,j) 在压缩数组中的下标 k 是多少？** 对“分布有规律”的矩阵，k 可以用公式直接算出来；对“分布无规律”的稀疏矩阵，只能把位置一起存下来。

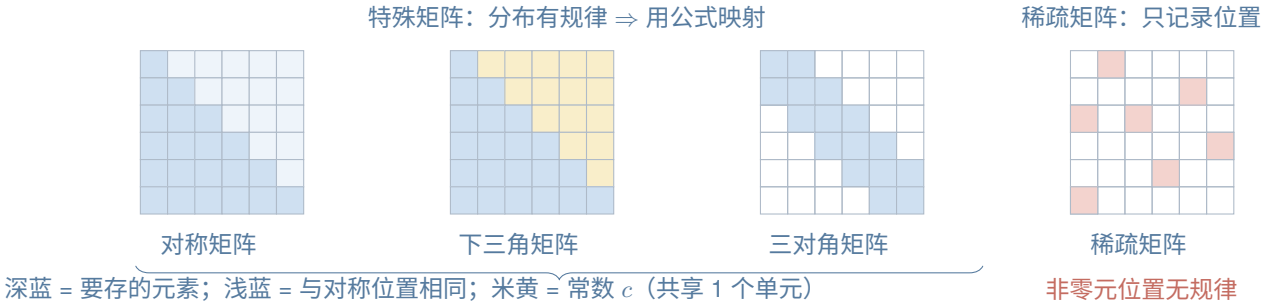


图 2.2　特殊矩阵（课件第 21 页）与稀疏矩阵的根本区别：非零元的分布是否有规律

2.2　对称矩阵

它解决什么问题

在 n 阶对称矩阵中，a_{ij} 和 a_{ji} 总相等，存两份是纯粹的浪费。只存“一半”就够了，难点在于：存一半以后，怎样还能取到另一半？

❑ 对称矩阵（笔记第 4 页）

若 n 阶方阵 A 满足 $\boxed{a_{ij}=a_{ji}}\ (1\le i,j\le n)$，则称 A 为对称矩阵。

存储方法：只存储下三角（或上三角）**包括主对角线**的元素，共 $\dfrac{n(n+1)}{2}$ 个。

正例：笔记第 5 页的 5 阶矩阵（下图左）沿主对角线“折叠”后完全重合。**反例：** $\begin{pmatrix}1&2\\3&1\end{pmatrix}$ 中 $a_{12}=2\neq a_{21}=3$，不是对称矩阵，只存一半就会丢信息。

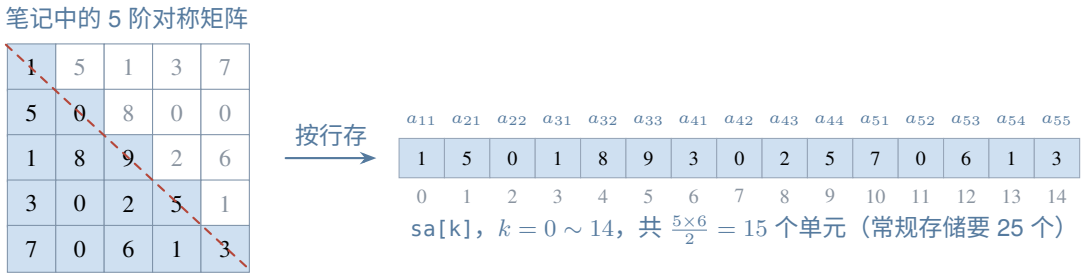


图 2.3　对称矩阵只存下三角（深色），按行序依次放入一维数组 sa

为什么是 $\frac{n(n+1)}{2}$ 个？

下三角（含对角线）第 1 行有 1 个元素，第 2 行 2 个，……，第 n 行 n 个，共

$$1+2+\cdots+n=\frac{n(n+1)}{2}\quad\text{（笔记第 5 页顶部的“}1+2+\cdots+n\text{”）.}$$

上三角同样是 $\frac{n(n+1)}{2}$ 个，所以存上三角还是存下三角，**占用空间一样**。

核心公式：a_{ij} 在 sa 中的下标

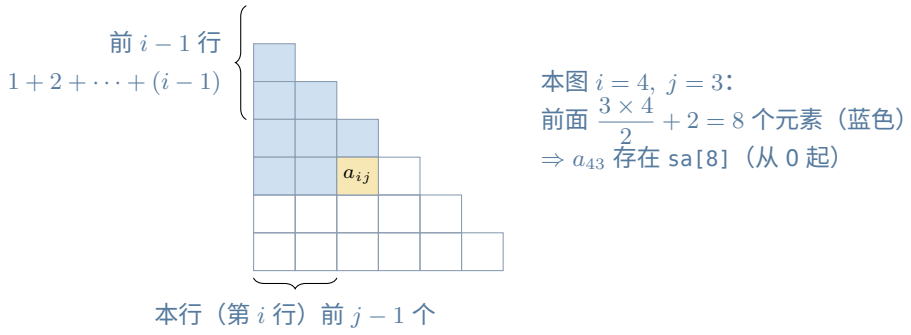


图 2.4　$i \ge j$ 时，a_{ij} 前面的元素 = 前 $i-1$ 行的“阶梯” + 本行前 $j-1$ 个

Σ 对称矩阵压缩下标（课件第 22 页；sa 下标从 0 开始）

$$k=\begin{cases}\dfrac{i(i-1)}{2}+j-1, & i\ge j\ \text{（在下三角，直接算）}\\[2ex] \dfrac{j(j-1)}{2}+i-1, & i<j\ \text{（在上三角，用}a_{ij}=a_{ji}\text{交换}i,j\text{）}\end{cases}$$

其中 $0 \le k \le \frac{n(n+1)}{2}-1$。笔记第 5 页 ④：“$a_{ij}$ 前有 $\frac{i(i-1)}{2}+j-1$ 个元素”——下标从 0 开始时，“前面有几个”就等于下标 k。

推导

已知： $i \ge j$，下三角按行序存入 `sa[0..]`。

第一步： 第 $1 \sim i-1$ 行全部排在 a_{ij} 之前，第 r 行有 r 个元素，共 $1+2+\cdots+(i-1)=\frac{(i-1)i}{2}$ 个。

第二步： 第 i 行中 $a_{i1},\ldots,a_{i,j-1}$ 排在它前面，共 $j-1$ 个。

第三步： 前面共 $\frac{i(i-1)}{2}+j-1$ 个；sa 从 0 开始，所以 k 就是这个数。

$i < j$ **的情形：** a_{ij} 在上三角，没有存。但 $a_{ij} = a_{ji}$，而 a_{ji}（行号 j 大于列号 i）在下三角，于是把上式中的 i, j 对调即可。 □

边界检验： $a_{11} \to 0$；$a_{nn} \to \frac{n(n-1)}{2} + n - 1 = \frac{n(n+1)}{2} - 1$，恰是最后一个单元。✓

例 2.1　用公式查元素

对图 2.3 的矩阵，求 $a_{4,3}$ 与 $a_{2,4}$ 在 sa 中的下标和值。

$a_{4,3}$：$i = 4 \geq j = 3$，$k = \frac{4\times3}{2} + 3 - 1 = 8$，sa[8] = 2；查原矩阵第 4 行第 3 列确为 2。✓

$a_{2,4}$：$i = 2 < j = 4$，交换：$k = \frac{4\times3}{2} + 2 - 1 = 7$，sa[7] = 0；原矩阵 $a_{24} = 0$。✓

本题想说明： 上三角的元素“借用”对称位置的下三角元素，所以 a_{24} 和 a_{42} 共用 sa[7]。

课程代码（文件夹 对称矩阵.cpp）中的取元素运算符，正是上面的公式：

C++　对称矩阵.cpp（节选）：operator() 把 (row,col) 映射到 elems 下标

```cpp
template <class ElemType>
ElemType &SymmtryMatrix<ElemType>::operator()(int row, int col)
{
    if (row < 1 || row > order || col < 1 || col > order)
    {   //出现异常
        cout << "下标出界!" << endl;
        exit(2);
    }
    if (row >= col) return elems[row * (row - 1) / 2 + col - 1];   //元素在下三角中
    else return elems[col * (col - 1) / 2 + row - 1];              //元素在上三角中
}
```

构造函数中分配的空间是 new ElemType[order * (order + 1) / 2]，即 $\frac{n(n+1)}{2}$ 个。下面的小程序独立实现同一公式，并对全部 25 个位置逐一核对：

C++　04_symmetric.cpp：压缩并验证全部位置

```cpp
int SymIndex(int i, int j)                  // 1 起始的 (i,j) -> sa 的下标 k
{
    if (i >= j) return i * (i - 1) / 2 + j - 1;   // 在下三角：直接算
    else        return j * (j - 1) / 2 + i - 1;   // 在上三角：a(i,j)=a(j,i)，交换
}
int main()
{
    const int n = 5;
    int b[n][n] = { {1,5,1,3,7}, {5,0,8,0,0}, {1,8,9,2,6}, {3,0,2,5,1}, {7,0,6,1,3} };
    int sa[n * (n + 1) / 2];                // 15 个单元
    for (int i = 1; i <= n; i++)            // 压缩：只取 i>=j 的元素
        for (int j = 1; j <= i; j++)
            sa[SymIndex(i, j)] = b[i - 1][j - 1];
    for (int k = 0; k < n * (n + 1) / 2; k++) cout << sa[k] << " ";
    cout << endl;
    bool ok = true;                         // 用全部 25 个位置检验映射
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (sa[SymIndex(i, j)] != b[i - 1][j - 1]) ok = false;
    cout << (ok ? "全部 25 个元素还原正确" : "错误") << endl;
    cout << "a(2,4) -> k = " << SymIndex(2, 4) << endl;
    return 0;
```

```
23 }
```

运行输出（g++ 实测）

```
1 5 0 1 8 9 3 0 2 5 7 0 6 1 3
全部 25 个元素还原正确
a(2,4) -> k = 7
```

读代码：第 11–13 行只遍历 $j \le i$ 的位置，把下三角“搬”进 sa；第 17–19 行遍历所有 25 个位置，用公式取回来与原矩阵比较，说明上三角的元素也能正确取到。（完整程序含 #include 等，见随附源文件。）

✘ 易错点

错误 1：$i < j$ 时忘了交换，直接代 $\frac{i(i-1)}{2} + j - 1$。例如 $a_{2,4}$ 会算成 $1+3=4$，取到的是 $a_{3,2}$。

错误 2：把“第几个”和“下标”混为一谈。sa 从 0 开始时 k = 前面个数；若题目说存入 $B[1..]$，则位置 = 前面个数 $+1$。

错误 3：以为存上三角比存下三角省空间——两者都是 $\frac{n(n+1)}{2}$。

★ 生活中的它：九九乘法表

生活场景：小学背的九九乘法表只印“一一得一”到“九九八十一”共 45 句，没有“二一得二”。

对应关系：9×9 的乘积矩阵 $a_{ij} = i \times j$；乘法交换律 $i \times j = j \times i$ 就是 $a_{ij} = a_{ji}$（对称）；口诀表只印 $i \le j$ 的一半（含对角线 $i = j$），正是“只存一个三角”。

小估算：$\frac{9\times10}{2} = 45$ 句，比 81 少了 36 句。查“七八”时，口诀表里只有“七八五十六”（小数在前），这就是 $i < j$ 时“交换下标去查”。

边界说明：真实的口诀表是为了好背，并不是为了省纸；类比只在“利用对称只记一半”这一点上成立。

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

⟶ 抽象模型：对称矩阵 $a_{ij} = i \cdot j$
只存上三角（含对角线）45 个
查 $a_{8,7} \Rightarrow$ 交换后查 $a_{7,8} = 56$

图 2.5　生活场景 ↔ 抽象模型：口诀表只保留米色的一半

➢ 对照：本章的对称矩阵压缩 ↔ 线性代数中的对称矩阵（推广型 / 同名异义）

一句话定位：两门课说的是同一个“对称矩阵”，但数据结构关心“怎么省空间地存”，线性代数关心“有什么运算性质”。

比较维度	线性代数	数据结构（本章）
定义	$A^{\mathrm{T}} = A$	$a_{ij} = a_{ji}$（同一回事）
关注点	特征值为实数、可正交对角化等	只存 $\frac{n(n+1)}{2}$ 个元素，公式映射
下标	一般从 1 开始	C++ 数组从 0 开始，公式中常见 -1

迁移判断（把线性代数的运算结果用本章方法存起来）：

结论	迁移	理由或反例
两个对称矩阵的和 $A+B$ 仍可压缩存储	✓	$(A+B)^{\mathrm{T}} = A+B$，仍对称；直接把两个 sa 逐项相加即可
数乘 kA 仍可压缩存储	✓	仍对称
两个对称矩阵的积 AB 仍可压缩存储	✗	反例：$\left(\begin{smallmatrix}1&2\\2&3\end{smallmatrix}\right)\left(\begin{smallmatrix}0&1\\1&0\end{smallmatrix}\right) = \left(\begin{smallmatrix}2&1\\3&2\end{smallmatrix}\right)$ 不对称；只有 $AB = BA$ 时才对称

2.3　三角矩阵

❑ 三角矩阵（课件第 25 页 / 笔记第 5 页）

下三角矩阵：主对角线**以上**的元素（不包括对角线）全部为同一个常数 c（c 常为 0）；
上三角矩阵：主对角线**以下**的元素（不包括对角线）全部为同一个常数 c。
存储方法：存下（上）三角中的 $\frac{n(n+1)}{2}$ 个元素，重复元素 c **共享一个存储空间**，共占 $\boxed{\frac{n(n+1)}{2}+1}$ 个单元。



图 2.6　三角矩阵：蓝色 $\frac{n(n+1)}{2}$ 个元素逐一存，米黄的常数 c 只存一次

下三角矩阵（课件第 25 页的约定：sa[0] 存 c）

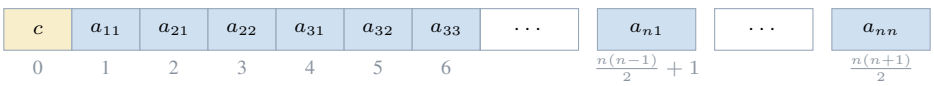


图 2.7　课件约定：常数 c 放在 sa[0]，下三角元素从 sa[1] 起按行存放

Σ 下三角矩阵压缩下标（课件第 25 页；sa[0] 存 c）

课件：$$k=\begin{cases}\dfrac{i(i-1)}{2}+j, & i\geq j\\ 0, & i<j\text{（常数}c\text{）}\end{cases}$$

笔记：$$k=\begin{cases}\dfrac{i(i-1)}{2}+j, & i\geq j\\ \dfrac{n(n+1)}{2}+1, & i<j\end{cases}$$

笔记第 6 页的约定是：元素存入 $B[1..]$，常数 c 放在最后一个位置。

为什么与对称矩阵公式差一个 1？ 前面元素个数仍是 $\frac{i(i-1)}{2}+j-1$；课件把 c 放在 `sa[0]`，所有元素整体右移一格，所以下标 $+1$；笔记用 1 起始的 $B[1..]$，“第几个”本来就比“前面个数”多 1。两种约定对 $i\geq j$ 恰好给出同一个数，区别只在 c 放在哪。

C++　下三角矩阵.cpp（节选）：课程代码中的映射

```cpp
template <class ElemType>
ElemType &LowerTriangularMatrix<ElemType>::operator()(int row, int col)
{
    if (row < 1 || row > order || col < 1 || col > order)
    {   //出现异常
        cout << "下标出界!" << endl;
        exit(2);
    }
    if (row >= col) return elems[row * (row - 1) / 2 + col];   //元素在下三角中
    else return elems[0];                                      //元素在其他位置
}
// 构造函数中: elems = new ElemType[order * (order + 1) / 2 + 1];
```

运行文件夹中的程序（4 阶下三角矩阵，$c=0$），输出与原矩阵一致：

运行输出（g++ 实测）

```
9       0       0       0
5       3       0       0
6       1       2       0
8       4       3       4
```

✘ 易错点：给“常数区”赋值会改掉所有 c

因为右上方的所有位置都返回同一个 `elems[0]` 的引用，执行 `a(1,3) = 7;` 会把整个右上方都变成 7。课程程序的 `main` 把原矩阵逐个赋值进去，之所以没出问题，是因为右上方全是 0，反复写入的都是 0。

上三角矩阵（笔记第 5 页、课件第 57 页课堂题）

上三角按行存时，每一行的元素个数不再是 $1,2,3,\ldots$，而是 $n,n-1,n-2,\ldots$：

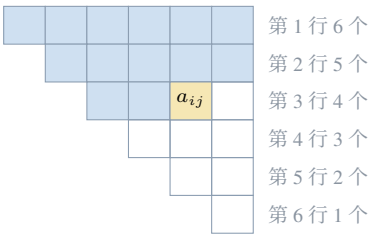


图 2.8　上三角按行存：第 r 行有 $n-r+1$ 个元素，而且第 i 行从 a_{ii} 开始

推导：上三角矩阵 a_{ij}（$i \le j$）前面有多少个元素

第一步：第 r 行存 $a_{rr}, \ldots, a_{rn}$，共 $n-r+1$ 个。前 $i-1$ 行共

$$\sum_{r=1}^{i-1}(n-r+1) = n+(n-1)+\cdots+(n-i+2) = \frac{(i-1)\,[\,n+(n-i+2)\,]}{2} = \frac{(i-1)(2n-i+2)}{2}.$$

（等差数列求和：项数 $i-1$，首项 n，末项 $n-i+2$。）

第二步：第 i 行从 a_{ii} 开始，a_{ij} 前面是 $a_{ii}, \ldots, a_{i,j-1}$，共 $j-i$ 个（**不是 $j-1$ 个**）。

结论：前面共 $\frac{(i-1)(2n-i+2)}{2} + j - i$ 个。 □

Σ 上三角矩阵压缩位置

笔记 / 课件第 57 页（存入$B[1..]$）：
$$\boxed{k = \frac{(i-1)(2n-i+2)}{2} + j - i + 1} \quad (i \le j)$$

sa 从 0 起（不含c）：
$$k = \frac{(i-1)(2n-i+2)}{2} + j - i \quad (i \le j)$$

笔记的约定中，$i > j$ 时为常数 c，存在 $B\left[\frac{n(n+1)}{2}+1\right]$。

例 2.2　上三角矩阵定位（$n=6$，$B[1..]$ 约定）

求 $a_{3,5}$ 与 $a_{5,2}$ 的位置。

$a_{3,5}$：$i=3 \le j=5$，$k = \frac{2\times(12-3+2)}{2} + 5 - 3 + 1 = 11 + 3 = 14$（与图 2.8 数出的“第 14 个”一致）。

$a_{5,2}$：$i > j$，是常数 c，位置 $\frac{6\times7}{2} + 1 = 22$。

下面的程序把公式与“逐个数”的结果对比，确认两种三角矩阵公式都对：

C++　05_triangular.cpp（节选）

```cpp
// 课件/文件夹代码约定：sa[0] 存常数 c，下三角元素从 sa[1] 开始按行存
int LowerIndex(int i, int j)
{
    if (i >= j) return i * (i - 1) / 2 + j;
    else        return 0;                       // 上方的常数 c
}
// 笔记约定：B[1..n(n+1)/2] 按行存上三角，常数 c 放在 B[n(n+1)/2+1]
int UpperPos(int i, int j, int n)
{
    if (i <= j) return (i - 1) * (2 * n - i + 2) / 2 + j - i + 1;
    else        return n * (n + 1) / 2 + 1;
```

```cpp
}
int main()
{
    const int n = 6;
    int pos = 0;                                    // 逐个数位置，检验公式
    bool ok = true;
    for (int i = 1; i <= n; i++)
        for (int j = i; j <= n; j++)
            if (UpperPos(i, j, n) != ++pos) ok = false;
    int k = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= i; j++)
            if (LowerIndex(i, j) != ++k) ok = false;
    cout << (ok ? "两个公式与逐个计数一致" : "错误") << endl;
    cout << "n=6: 上三角 a(3,5) 在 B[" << UpperPos(3, 5, n) << "], "
         << "a(5,2) 在 B[" << UpperPos(5, 2, n) << "]" << endl;
    cout << "下三角 a(3,2) 在 sa[" << LowerIndex(3, 2) << "]" << endl;
    return 0;
}
```

运行输出（g++ 实测）

```
两个公式与逐个计数一致
n=6: 上三角 a(3,5) 在 B[14], a(5,2) 在 B[22]
下三角 a(3,2) 在 sa[5]
```

为什么 C++ 中 `(i-1)*(2*n-i+2)/2` 不会因整数除法出错？ $(i-1)$ 与 $(2n-i+2)$ 的和为 $2n+1$（奇数），所以两者一奇一偶，乘积必为偶数，先乘后除没有余数。同理 $i(i-1)$ 是相邻整数之积，必为偶数。但如果写成 `(i-1)/2*(2*n-i+2)` 先做除法，就会截断出错——**务必先乘后除**。

情形	存储约定	a_{ij} 的下标 / 位置	常数 c
对称矩阵（课件 22 页）	`sa[0..]`，无 c	$\frac{i(i-1)}{2}+j-1$（$i\ge j$）；$\frac{j(j-1)}{2}+i-1$（$i<j$）	无
下三角（课件 25 页、代码）	`sa[0]`= c，元素从 1 起	$\frac{i(i-1)}{2}+j$（$i\ge j$）	`sa[0]`
下三角（笔记）	$B[1..]$，c 在最后	$\frac{i(i-1)}{2}+j$（$i\ge j$）	$B[\frac{n(n+1)}{2}+1]$
上三角（笔记、课件 57 页）	$B[1..]$，c 在最后	$\frac{(i-1)(2n-i+2)}{2}+j-i+1$（$i\le j$）	$B[\frac{n(n+1)}{2}+1]$
上三角（0 起，不含 c）	`sa[0..]`	$\frac{(i-1)(2n-i+2)}{2}+j-i$（$i\le j$）	另行约定

表 2.1　三角类矩阵公式总对照：先看清“下标从几开始、c 放在哪”，再选公式

★ 生活中的它：每月累计存款

生活场景：每月往存钱罐里放一笔钱，第 i 个月末的累计金额 = 前 i 个月每月存入之和。**对应关系：**累计金额 $s_i=\sum_{j=1}^{n}a_{ij}x_j$，其中 x_j 是第 j 月存入的钱；第 i 个月的累计不会用到“未来”的月份，所以 $j>i$ 时系数 $a_{ij}=0$——系数矩阵是**下三角矩阵**，常数 $c=0$。**小估算：**12 个月的系数矩阵有 144 个位置，压缩只需 $\frac{12\times13}{2}+1=79$ 个单元。

边界说明：这个矩阵的下三角元素全是 1，实际编程只需一个累加循环；类比的是“未来不影响过去”造成的三角形状。

➢ 对照：三角矩阵压缩 ↔ 线性代数中的三角矩阵（同名异义 / 推广型）

一句话定位：线性代数的三角矩阵要求另一侧**全为 0**；本章允许另一侧是**任意同一个常数** c，是更宽的概念（$c=0$ 时二者一致）。

线性代数中的结论	迁移	理由或反例
两个下三角矩阵的乘积仍是下三角	△	只在 $c=0$ 时成立；$c\neq 0$ 时如 $\left(\begin{smallmatrix}1&1\\1&1\end{smallmatrix}\right)^2=\left(\begin{smallmatrix}2&2\\2&2\end{smallmatrix}\right)$，右上不再是常数 1
行列式 = 主对角线元素之积	△	同样只在 $c=0$ 时成立
下三角矩阵的转置是上三角矩阵	✓	存储时要换用上三角公式，不能沿用下三角公式

同一个小算例：$n=3$、$c=0$ 的下三角矩阵 $\left(\begin{smallmatrix}1&0&0\\2&3&0\\4&5&6\end{smallmatrix}\right)$：线性代数里它的行列式 $=1\times3\times6=18$；数据结构里它存为 sa $=[0,1,2,3,4,5,6]$，共 $\frac{3\times4}{2}+1=7$ 个单元。

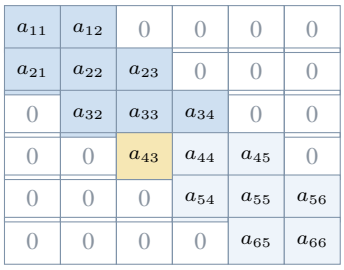
2.4　三对角矩阵（对角矩阵）

❑ 对角矩阵 / 三对角矩阵（笔记第 6 页 / 课件第 28 页）

对角矩阵（带状矩阵）：在 $n\times n$ 的方阵中，所有非 0 元素都集中在以主对角线为中心的带状区域中，区域外的值为 0（或同一常数 c）。

三对角矩阵：带宽为 3，即只有满足 $\boxed{|i-j|\le 1}$ 的元素可能非零（主对角线和紧挨它的上、下两条对角线）。非零区域共 $\boxed{3n-2}$ 个元素。

为什么是 $3n-2$：主对角线 n 个，上、下次对角线各 $n-1$ 个，$n+2(n-1)=3n-2$。也可以按行数：第 1 行和第 n 行各 2 个，中间 $n-2$ 行各 3 个，$4+3(n-2)=3n-2$。



按行存，sa[0] 存 0/c（课件第 28 页）：
第 1 行 2 个，第 2、3 行各 3 个 ⇒ 前 3 行 8 个；
第 4 行从 a_{43} 开始，a_{43} 前面共 8 个元素
⇒ a_{43} 在 sa[8+1]=sa[9]；公式 $2(4-1)+3=9$ ✓

图 2.9　6 阶三对角矩阵，按行压缩（深蓝 = 排在 a_{43} 前面的元素）

Σ 三对角矩阵按行压缩（课件第 28 页；sa[0] 存 0/c）

$$\boxed{k=2(i-1)+j,\qquad |i-j|\le 1}$$

sa 共 $3n-2+1$ 个单元（含 sa[0]），最后一个元素 a_{nn} 在 $k=2(n-1)+n=3n-2$。

推导（教学补全：课件只给结论）

第一步：第 1 行有 2 个元素；第 $2 \sim i-1$ 行每行 3 个。$i \geq 2$ 时，前 $i-1$ 行共 $2+3(i-2) = 3i-4$ 个（$i = 1$ 时为 0，下式同样适用）。

第二步：第 i 行的非零元从 $a_{i,i-1}$ 开始（$i = 1$ 时从 a_{11} 开始），a_{ij} 前面有 $j-(i-1) = j-i+1$ 个（$i = 1$ 时为 $j-1$ 个）。

第三步： $i \geq 2$ 时，前面共 $(3i-4)+(j-i+1) = 2i+j-3$ 个；`sa[0]` 被常数占用，下标再加 1：$k = 2i+j-2 = 2(i-1)+j$。

$i = 1$ 时：前面 $j-1$ 个，$k = j$；而 $2(1-1)+j = j$，公式同样成立。 □

反过来由 k 求 (i,j)（拓展）：$i = \lfloor (k+1)/3 \rfloor + 1$，$j = k - 2(i-1)$（$1 \leq k \leq 3n-2$）。例如 $k = 9$：$i = \lfloor 10/3 \rfloor + 1 = 4$，$j = 9-6 = 3$。

另一种映射：课程代码按“对角线”存放

文件夹中的 **三对角矩阵.cpp** 用的不是按行映射，而是先存高对角线、再存主对角线、最后存低对角线（共 $3n-1$ 个单元，`elems[0]` 存常数）：

C++　三对角矩阵.cpp（节选）：按对角线映射

```cpp
template <class ElemType>
ElemType &TriDiagonalMatrix<ElemType>::operator()(int row, int col)
{
    if (row < 1 || row > order || col < 1 || col > order)
    {   //出现异常
        cout << "下标出界!" << endl;
        exit(2);
    }
    if (row - col == 1) return elems[2 * order + row - 2];     //元素在低对角线上
    else if (row - col == 0) return elems[order + row - 1];    //元素在主对角线上
    else if (row - col == -1) return elems[row];               //元素在高对角线上
    else return elems[0];                                      //元素在其他位置
}
// 构造函数中: elems = new ElemType[3 * order - 1];
```



图 2.10　课程代码的对角线映射（$n = 5$，共 $3n-1 = 14$ 个单元）；同一个 a_{43} 在这里是 `elems[12]`，按行映射却是 `sa[9]`

运行输出（g++ 实测）

```
映射一正确，最后下标 = 13 (3n-2 = 13)
a(4,3): 映射一 sa[9]，映射二 elems[12]
```

解释：以上是 `06_tridiagonal.cpp` 的输出，它同时实现了两种映射（源码随附）。**两种映射都对，只是约定不同**：按行映射便于“一行一行处理”；按对角线映射的公式更整齐。考试题若给了映射规则，就严格按题目的规则算。

★ 生活中的它：一条地铁线的“相邻站”关系

生活场景：一条不分叉的地铁线有 n 个站，列车只在相邻两站之间直接运行。
对应关系：站 = 行号/列号；“第 i 站与第 j 站之间的区间运行时间”a_{ij} 只在 $|i-j|=1$ 时有意义，主对角线放本站停靠时间，其余为 0——这是一个三**对角矩阵**。
小估算：30 个站：常规存储 900 个单元，按行压缩只需 $3\times30-2+1=89$ 个。
边界说明：有换乘、有环线的线路网会出现带外的非零元，就不再是三对角，而更接近第 7 章的“图”。

➢ 对照：三对角矩阵 ↔ 大学物理中的“弹簧—质点链”（类比同构型，拓展联想）

拓展联想：以下内容只用于建立知识联系，当前阶段不要求掌握。
一句话定位：n 个质点用弹簧串成一条链，每个质点只受左右两根弹簧的力，列出的方程组系数矩阵就是三对角的；一维热传导、RC 梯形电路的节点方程也一样。
相同之处：“只与相邻的发生关系”$\Leftrightarrow$ 只有 $|i-j|\le1$ 的元素非零。
不同之处及根源：本章只关心“怎么存”；数值计算课程还会利用这个结构，用“追赶法”以 $O(n)$ 的时间解方程组（普通消元法是 $O(n^3)$）。

结论	迁移	理由
只存带内 $3n-2$ 个元素不丢信息	✓	带外全为 0
物理方程的系数矩阵通常还对称	△	只有作用力对等（如弹簧链）时才对称，此时还可再省一条次对角线

2.5　稀疏矩阵

它解决什么问题

对称、三角、三对角矩阵的非零元位置有规律，所以能用公式 $k=f(i,j)$。但许多矩阵的非零元很少、位置却没有规律：

课件第 31 页：$A_{6\times7}$

0	0	0	22	0	0	15
0	11	0	0	0	17	0
0	0	0	-6	0	0	0
0	0	0	0	0	39	0
91	0	0	0	0	0	0
0	0	28	0	0	0	0

42 个元素中只有 8 个非零（粉色），位置看不出规律，无法写出 $k=f(i,j)$。

常规存储的问题（课件第 32 页）：
① 零值元素占了很大空间；
② 计算中进行了很多和零值的运算，遇除法还要判别除数是否为零。

图 2.11　稀疏矩阵：非零元少且分布没有规律

❑ 稀疏矩阵（课件第 31 页 / 笔记第 6 页）

稀疏矩阵是非零元素个数远远少于矩阵元素个数的矩阵。

稀疏因子（笔记）：设 $m \times n$ 矩阵中有 t 个非零元素，令 $\boxed{\delta = \dfrac{t}{m \times n}}$；通常当 $\delta \le 0.05$ 时称为稀疏矩阵（经验标准）。

例：图 2.11 中 $\delta = \frac{8}{42} \approx 0.19$。按 0.05 的经验标准它并不“稀疏”，它只是课件用来演示方法的小例子；真正的稀疏矩阵往往有成千上万行。

稀疏矩阵的基本操作（课件第 34–35 页）：`GetRows()` 返回行数；`GetCols()` 返回列数；`GetNum()` 返回非零元个数；`Empty()` 判断是否为空；`SetElem(r,c,v)` 设置指定位置的元素值；`GetElem(r,c,v)` 求指定位置的元素值。

存储方式（课件第 36 页）：顺序压缩存储——**三元组表示法**；链式压缩存储——**带行指针的链表**、十字链表（课件标注“不讲”）。

2.5.1　三元组顺序表

❑ 三元组与三元组顺序表（课件第 37 页 / 笔记第 6–7 页）

对每个非零元素，用一个**三元组** (i, j, a_{ij})——（行号，列号，元素值）——唯一确定它。以**顺序表**存储这些三元组（非零元按**行序**有序存放），就得到稀疏矩阵的顺序存储结构——**三元组顺序表**。

为避免丢失信息，通常再增设一个**总体信息**元组：（行数，列数，非零元素个数）。

为什么 / 直观理解

为什么必须记“总体信息”？只看非零元，你无法知道矩阵有多大。例如课件例子的最后一列全是 0、第 4 行全是 0，如果不记“5 行 6 列”，还原时就会以为它是 5×6 以外的尺寸，或者漏掉全零的行列。

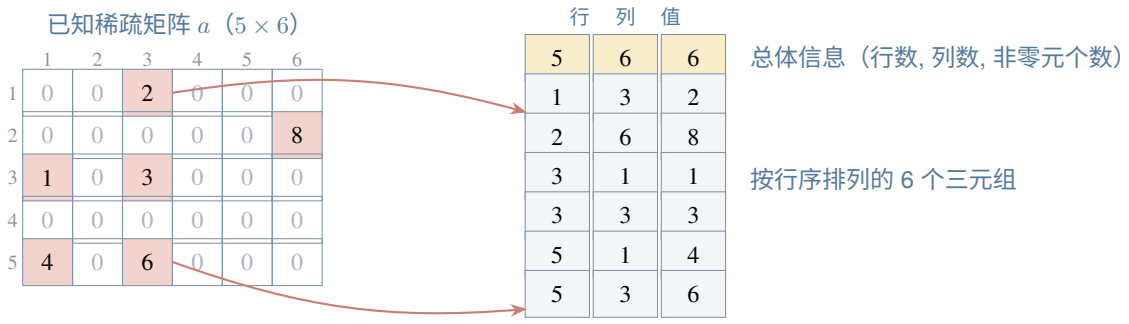


图 2.12　课件第 37 页：稀疏矩阵 → 三元组顺序表（首行是总体信息）

例 2.3（笔记第 7 页，已纠错）　矩阵 → 三元组；三元组 → 矩阵

① 转换为三元组表：$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 \\ 15 & 0 & 0 & 0 & 0 \end{pmatrix}$　② 还原的结果：$\begin{pmatrix} 0 & 2 & 0 & 0 \\ 12 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 6 & 0 \\ 16 & 0 & 0 & 0 \end{pmatrix}$

① **解：**逐行从左到右扫描非零元：总体信息 $(6,5,7)$；三元组依次为 $(1,2,12),(1,3,9),(3,1,-3),(3,4,14),(4,3,24),(5,2,18),(6,1,15)$。

② **题目：**总体信息 $(6,4,6)$，三元组 $(1,2,2),(2,1,12),(3,1,3),(4,4,4),(5,3,6),(6,1,16)$，还原稀疏矩阵。

② **解：**先画出 6 行 4 列的全 0 矩阵，再按三元组逐个填入，得到上方右边的矩阵。

检查：填入的非零元正好 6 个，与总体信息一致。✓　**本题想说明：**还原时必须先看总体信息定尺寸，否则第 4 列是否存在都无法确定。

C++ 实现：数据成员与 SetElem

课件第 38–40 页与文件夹 稀疏矩阵 _ 三元组排序.cpp 定义了三元组结构和三元组顺序表类：

C++　稀疏矩阵 _ 三元组排序.cpp（节选）：数据成员

```cpp
template<class ElemType>
struct Triple
{
    int row, col;                       //非零元素的行下标与列下标
    ElemType value;                     //非零元素的值
    Triple();
    Triple(int r, int c, ElemType v);
};

template<class ElemType>
class TriSparseMatrix
{
protected:
    Triple<ElemType> *triElems;         //存储稀疏矩阵的三元组表
    int maxSize;                        //非零元素最大个数
    int rows, cols, num;                //稀疏矩阵的行数、列数及非零元个数
public:
    // GetRows / GetCols / GetNum / Empty / SetElem / GetElem / SimpleTranspose ...
};
```

`rows, cols, num` 就是“总体信息”；`triElems` 数组按行序存放三元组。最值得读懂的是 `SetElem`——它要在**保持行序有序**的前提下完成“修改 / 删除 / 插入”三件事：

C++　稀疏矩阵 _ 三元组排序.cpp（节选）：SetElem

```cpp
template <class ElemType>
bool TriSparseMatrix<ElemType>::SetElem(int r, int c, const ElemType &v)
{
    if (r > rows || c > cols || r < 1 || c < 1)
        return false;                                   //下标范围错
    int tempPos, pos;
    for (pos = num - 1; pos >= 0 &&
        (r < triElems[pos].row || r == triElems[pos].row && c < triElems[pos].col);
        pos--);                                         //从后往前找三元组位置
    if (pos >= 0 && triElems[pos].row == r && triElems[pos].col == c)
    {   //找到三元组
        if (v == 0)
        {   //删除三元组：后面的元素依次前移
            for (tempPos = pos + 1; tempPos < num; tempPos++)
                triElems[tempPos - 1] = triElems[tempPos];
            num--;
        }
        else triElems[pos].value = v;                   //修改元素值
        return true;
    }
    else if (v != 0)
    {   //未找到且 v!=0：插入到 pos+1
        if (num < maxSize)
        {
            for (tempPos = num - 1; tempPos > pos; tempPos--)
                triElems[tempPos + 1] = triElems[tempPos];  //后移元素
            triElems[pos + 1].row = r;
            triElems[pos + 1].col = c;
            triElems[pos + 1].value = v;
            num++;
            return true;
        }
        else return false;                              //溢出
    }
    else return true;                                   //未找到且 v==0：什么也不做
}
```

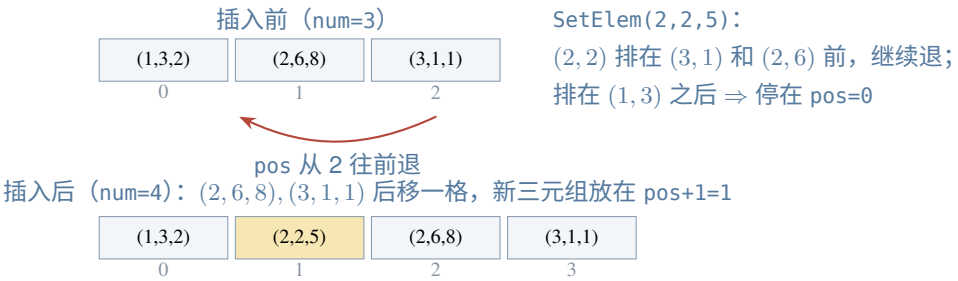


图 2.13　SetElem 的插入过程：先从表尾往前找位置，再把后面的三元组后移

三种情况：① 找到 (r,c) 且 $v\neq 0$：只改值；② 找到且 $v=0$：删除（后面前移，num--）——**三元组表里永远不存 0**；③ 没找到且 $v\neq 0$：插入到 pos+1（后面后移，num++），满了就返回 false。没找到且 $v=0$ 时什么也不做。

GetElem 则是从头到尾顺序查找 (r,c)，找到返回其值，找不到说明该元素为 0。

✓ 三元组顺序表的特点（笔记第 8 页）

优点：非零元在表中按行序有序存储，因此便于进行依行顺序处理的矩阵运算。

缺点：不能随机存取。若按行号存取某一行中的非零元，需从头开始查找（`GetElem` 是顺序查找，最坏要比较 `num` 次）。

★ 生活中的它：几乎空场的电影院

生活场景：工作日上午的电影只卖出几张票，售票系统不会存“每个座位是否有人”的整张表，而是存一份“已售订单”：（排，座，票价）。

对应关系：座位表 = 稀疏矩阵；已售订单 = 三元组；放映厅“几排几座” = 总体信息。订单按排号、座号排列 = 按行序有序。

小估算：200 个座位卖出 6 张：整张表要 200 个单元；三元组表要 $(6+1)\times 3 = 21$ 个整数。

边界说明：真实系统还要存时间、手机号等，且常用数据库而不是数组；类比的只是“只记录非空位置”。

2.5.2 稀疏矩阵的转置

为什么 / 直观理解

为什么不能“只把每个三元组的行列号对调”？对调后元素都对了，但**顺序乱了**：原来按行序排列，对调后变成按“原列号”排列，可原列号不是有序的。三元组顺序表要求按行序有序，所以必须重新安排顺序。

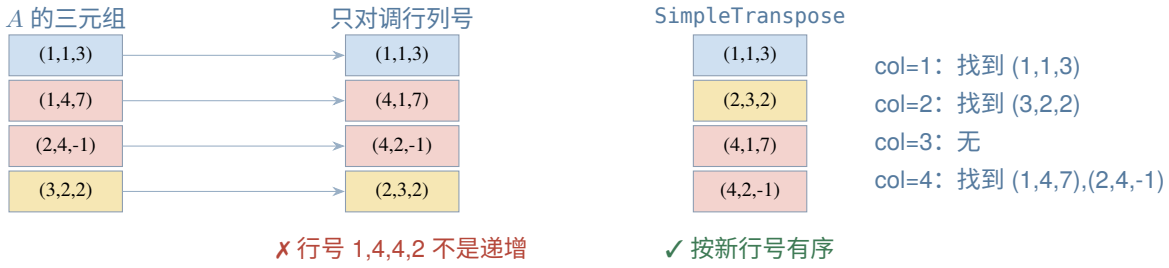


图 2.14　课件第 46 页的 $A_{3\times4}$：直接对调会乱序；简单转置按“原列号 $1,2,\ldots$”逐列扫描

❑ 简单转置算法的基本思想（课件第 42 页）

第一次从转置前的稀疏矩阵 `source` 中取出应该放到转置后 `dest` 中第一个位置的元素，行列号互换后放入 `dest` 的第一个位置；第二次取出应放到第二个位置的元素……如此进行，依次生成 `dest` 中的各元素。

由于转置后**列号变行号**，所以转置后元素的行序排列**实质上是原矩阵元素的列序排列**：`for(col = 最小列号; col <= 最大列号; col++)` 在 `source` 中从头查找所有列号为 `col` 的三元组，若有，则行列交换后依次存入 `dest` 中 `destPos` 所指位置，同时 `destPos` 加 1。

C++　稀疏矩阵 _ 三元组排序.cpp（节选）：SimpleTranspose

```cpp
template<class ElemType>
void TriSparseMatrix<ElemType>::SimpleTranspose(
    const TriSparseMatrix<ElemType> &source, TriSparseMatrix<ElemType> &dest)
{
    dest.rows = source.cols;                              //行数
    dest.cols = source.rows;                              //列数
    dest.num = source.num;                                //非零元素个数
    dest.maxSize = source.maxSize;                        //最大非零元素个数
    delete []dest.triElems;                               //释放存储空间
    dest.triElems = new Triple<ElemType>[dest.maxSize]; //分配存储空间
    if (dest.num > 0)
    {
        int destPos = 0;                                  //dest 的下一个三元组的存放位置
        for (int col = 1; col <= source.cols; col++)
        {   //转置前的列变为转置后的行
            for (int sourcePos = 0; sourcePos < source.num; sourcePos++)
            {   //查找第 col 列的三元组
                if (source.triElems[sourcePos].col == col)
                {
                    dest.triElems[destPos].row = source.triElems[sourcePos].col;   //列变行
                    dest.triElems[destPos].col = source.triElems[sourcePos].row;   //行变列
                    dest.triElems[destPos].value = source.triElems[sourcePos].value;
                    destPos++;
                }
            }
        }
    }
}
```

逐段解释：第 5–10 行处理“总体信息”：行数、列数对调，非零元个数不变，重新分配空间。第 14 行的外层循环按**原矩阵的列号** 1 ～`source.cols` 走；第 16 行的内层循环每次都把 `source` 的三元组从头扫一遍。因为 `source` 本身按行序有序，同一列的元素被找出来时行号是递增的，于是 `dest` 中同一行的元素列号也递增——`dest` 自动按行序有序。

时间复杂度（教学补全）：外层循环 `cols` 次，每次内层扫描 `num` 个三元组，共 $O(\text{cols}\times\text{num})$。当 num $\ll$ rows $\times$ cols 时，这比直接转置二维数组的 $O(\text{rows}\times\text{cols})$ 快；但若矩阵并不稀疏（num 接近 rows $\times$ cols），就变成 $O(\text{rows}\times\text{cols}^2)$，反而更慢。

★ 生活中的它：选课表的“转置”

生活场景：教务系统按学生存选课记录：（学号，课程号，成绩），按学号排序。期末老师要按课程打印名单。

对应关系：“学生 $\times$ 课程”成绩表 = 稀疏矩阵（每人只选几门课）；按课程打印 = 转置成“课程 $\times$ 学生”；按课程号 1、2、3……逐门扫描全部记录 = `SimpleTranspose` 的外层循环。

小判断：因为原记录按学号有序，逐门扫描时找出的学生也按学号有序——名单不用再排序。

边界说明：真实系统用数据库索引一次就能查出某门课的学生，不必每门课都扫描全部记录。

2.5.3　稀疏矩阵的相加与相乘

相加（课件第 45 页）：$C = A + B$（A, B, C 都是 $m \times n$）。与普通矩阵加法不同的是，三元组表示的稀疏矩阵在加法计算时，**只有在结果矩阵的元素不为 0 时**，才调用添加元素函数添加到和矩阵的三元组中（和矩阵也是一个三元组表）。例如 $a_{ij} = 5$、$b_{ij} = -5$ 时，和为 0，不加入。

相乘（课件第 46 页）：设 A 为 $m \times p$，B 为 $p \times n$，则 $C = A \times B$ 为 $m \times n$：

$$C[i][j] = \sum_{k=1}^{p} A[i][k] \times B[k][j], \qquad 1 \le i \le m,\ 1 \le j \le n.$$

同样，只有积矩阵的元素不为 0 时，才添加到积矩阵的三元组中。

$$A = \begin{pmatrix} 3 & 0 & 0 & 7 \\ 0 & 0 & 0 & -1 \\ 0 & 2 & 0 & 0 \end{pmatrix} \quad \times B = \begin{pmatrix} 0 & 1 \\ 2 & 0 \\ 3 & 4 \\ 0 & 0 \end{pmatrix} \quad = C = \begin{pmatrix} 0 & 3 \\ 0 & 0 \\ 4 & 0 \end{pmatrix}$$

$c_{12} = 3 \times 1 + 0 + 0 + 7 \times 0 = 3$
$c_{31} = 0 + 2 \times 2 + 0 + 0 = 4$
$c_{11} = 3 \times 0 + 7 \times 0 = 0$，不存
C 的三元组：$(3, 2, 2)$
$(1, 2, 3), (3, 1, 4)$

图 2.15　课件第 46 页的稀疏矩阵乘法：6 个结果元素中只有 2 个非零进入三元组表

下面的程序用课程的 `TriSparseMatrix` 类（只借助其公有的 `GetElem/SetElem`）完成乘法，并演示建立、插入、删除与转置：

C++　07_sparse.cpp（节选）：乘法与演示

```cpp
template <class ElemType>
void Multiply(TriSparseMatrix<ElemType> &A, TriSparseMatrix<ElemType> &B,
              TriSparseMatrix<ElemType> &C)
{
    ElemType x, y;
    for (int i = 1; i <= A.GetRows(); i++)
        for (int j = 1; j <= B.GetCols(); j++) {
            ElemType sum = 0;
            for (int k = 1; k <= A.GetCols(); k++) {
                A.GetElem(i, k, x);  B.GetElem(k, j, y);
                sum += x * y;
            }
            if (sum != 0) C.SetElem(i, j, sum);   // 积为 0 就不存
        }
}
// main 中（ShowSparse 是只为打印三元组而派生的类）：
    for (int c = 1; c <= 6; c++)                 // 故意按“列”的顺序写入，
        for (int r = 1; r <= 5; r++)             // SetElem 仍会保持按行有序
            s.SetElem(r, c, a[r - 1][c - 1]);
    s.ShowTriples();
    s.SetElem(2, 2, 5);  s.ShowTriples();         // 插入
    s.SetElem(2, 6, 0);  s.ShowTriples();         // 删除（置 0）
    cout << "SetElem(7,1,3) = " << s.SetElem(7, 1, 3) << endl;   // 下标出界
```

运行输出（g++ 实测）

```
(5,6,6) (1,3,2) (2,6,8) (3,1,1) (3,3,3) (5,1,4) (5,3,6)
(5,6,7) (1,3,2) (2,2,5) (2,6,8) (3,1,1) (3,3,3) (5,1,4) (5,3,6)
(5,6,6) (1,3,2) (2,2,5) (3,1,1) (3,3,3) (5,1,4) (5,3,6)
SetElem(7,1,3) = 0
A  : (3,4,4) (1,1,3) (1,4,7) (2,4,-1) (3,2,2)
A^T: (4,3,4) (1,1,3) (2,3,2) (4,1,7) (4,2,-1)
A*B: (3,2,2) (1,2,3) (3,1,4)
```

读输出：第 1 行：即使按列的顺序写入，三元组表仍按行序排列——这是 `SetElem` 插入时找位置的功劳。第 2、3 行：插入 $(2,2,5)$ 后 `num` 变 7，把 $(2,6)$ 置 0 后该三元组被删除。第 4 行：行号 7 超过 5 行，返回 `false`（输出 0）。后 3 行与图 2.14、图 2.15 的手算结果完全一致。

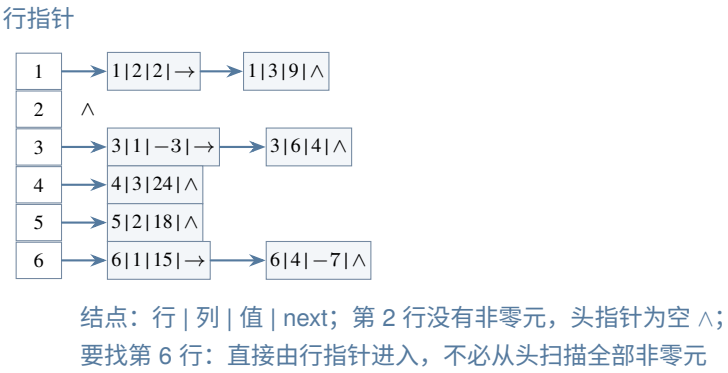
说明：这个乘法函数逐个位置调用 `GetElem`（顺序查找），是为了讲清思路的朴素写法，效率不高；课件只要求掌握“只存非零结果”的思想。

2.5.4　带行指针的链表

❑ 带行指针的链表（课件第 47 页）

把具有**相同行号**的非零元用一个单链表连接起来，稀疏矩阵中的若干行组成若干个单链表，再用一个**行指针数组**指向各行链表的头，合起来称为带行指针的链表。

$$M=\begin{pmatrix}0&2&9&0&0&0&0\\0&0&0&0&0&0&0\\-3&0&0&0&0&4&0\\0&0&24&0&0&0&0\\0&18&0&0&0&0&0\\15&0&0&-7&0&0&0\end{pmatrix}$$



结点：行 | 列 | 值 | next；第 2 行没有非零元，头指针为空 ∧；
要找第 6 行：直接由行指针进入，不必从头扫描全部非零元

图 2.16　课件第 47 页：稀疏矩阵 M 的带行指针的链表

与三元组顺序表比较：三元组顺序表找第 i 行要从头查；带行指针的链表可直接从第 i 个行指针进入该行。链表插入、删除非零元也不必移动其他元素。至于**十字链表**（每个结点再加列指针 `down`、行指针 `right`，同时挂在行链表和列链表上），课件标注“不讲”，知道它适合“非零元位置或个数经常变动”的情形即可。

➢ 对照：三元组 / 带行指针的链表 ↔ 第 7 章“图”的邻接矩阵与邻接表（思想重现型）

一句话定位：一张有 n 个顶点的图可以用 $n\times n$ 的邻接矩阵表示，但大多数图的边很少——邻接矩阵是稀疏矩阵，而“邻接表”正是“带行指针的链表”的思想。

	本章	第 7 章（后续）
行指针数组	每行一个头指针	每个顶点一个头指针
链表结点	(行, 列, 值)	(邻接顶点, 权值)
找第 i 行 / 顶点 i 的邻居	直接从第 i 个头指针进入	同左

本章结论	迁移	理由
零元不存，只存非零元	✓	没有边就不建结点
无向图的邻接矩阵可按对称矩阵压缩	✓	无向图中 $a_{ij} = a_{ji}$
有向图的邻接矩阵也可按对称矩阵压缩	✗	有向边 $i \to j$ 不意味着 $j \to i$

相近概念辨析

维度	对称矩阵	三角矩阵	稀疏矩阵
本质	$a_{ij} = a_{ji}$	一侧全为同一常数 c	非零元很少，位置无规律
存储单元数	$\frac{n(n+1)}{2}$	$\frac{n(n+1)}{2} + 1$（多一个 c）	$3 \times (t+1)$（三元组 + 总体信息）
怎样找 a_{ij}	公式；$i < j$ 时交换 i, j	公式；另一侧直接返回 c	查表（顺序查找）
随机存取	能	能	不能
易错	忘记交换	$j - i$ 写成 $j - 1$；c 的位置	漏写总体信息；存了 0

本章知识地图与小结

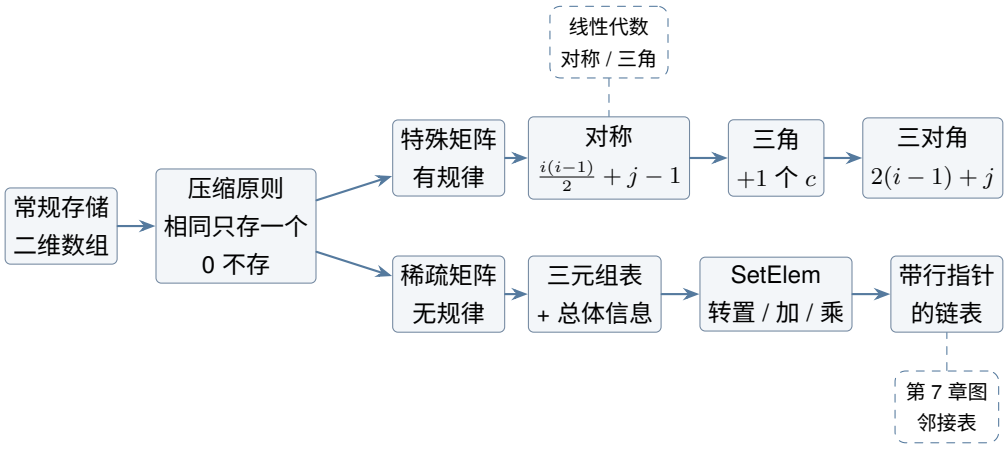


图 2.17　第 2 章知识地图

✓ 到这里，我们已经解决了什么？

有规律的矩阵（对称、三角、三对角）用“数前面有几个元素”推出 $k = f(i, j)$，既省空间又保留随机存取；无规律的稀疏矩阵用三元组（加总体信息）或带行指针的链表只记录非零元，代价是不能随机存取。转置要保持行序，所以按原列号逐列扫描；加、乘只把非零结果存入。

下一步还需要解决什么？到目前为止，数组和矩阵的元素都是“一个数”。如果允许元素本身又是一张表——表中有表——会得到什么结构？怎样描述它的“长度”和“层数”？这就是第 3 章的广义表。

学完这里，你应该能够回答

1. 压缩存储的两条原则是什么？特殊矩阵和稀疏矩阵的根本区别是什么？
2. 对称矩阵 $i<j$ 时为什么要交换 i,j？存上三角和存下三角哪个更省空间？
3. 课件的下三角公式 $\frac{i(i-1)}{2}+j$ 与对称矩阵公式为什么差 1？
4. 上三角矩阵第 i 行的元素从哪一列开始？为什么本行前面是 $j-i$ 个？
5. 三对角矩阵有多少个非零元？课件公式 $k=2(i-1)+j$ 中 $k=0$ 存的是什么？
6. 三元组表为什么要加总体信息？`SetElem(r,c,0)` 会做什么？
7. 为什么转置不能只对调行列号？简单转置的时间复杂度是多少？

第 3 章　广义表：表中有表（只讲概念）

本章回答三个问题：(1) 什么是广义表，它和线性表有什么不同？(2) 表头、表尾、长度、深度怎样求？(3) 广义表有哪些性质和基本操作？

知识路线：定义与记法 → 表头 / 表尾 → 长度 / 深度 → 四个性质 → ADT 与基本操作。课件第 59 页注明：本节**只讲简单的概念**，存储结构（课件第 67–77 页）**不讲**。

3.1　广义表的概念

它解决什么问题

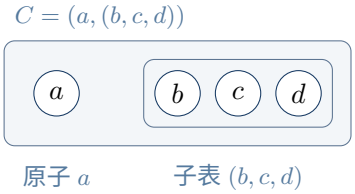
线性表的元素都是“同一层”的单个数据。可是很多数据天然是嵌套的：一本书有若干章，一章有若干节，节里还有小节；一个文件夹里既有文件，也有子文件夹。要描述“元素本身又是一张表”的结构，就需要把线性表推广——这就是广义表（General List，也称列表）。

❑ 广义表（课件第 60 页 / 笔记第 8 页）

广义表是 n $(n\ge 0)$ 个数据元素的有限序列，一般记作

$$LS=(a_1,a_2,\ldots,a_n).$$

其中 LS 是广义表的名称，a_i $(1\le i\le n)$ 是 LS 的**直接数据元素**（也称成员）。a_i 可以是单个的数据元素，也可以是一个广义表，分别称为 LS 的**单数据元素（或原子）**和**子表**。**习惯上**用**大写字母**表示广义表，**小写字母**表示原子。n 称为表的长度，$n=0$ 时为空表。



外框 = 广义表 C；里框 = 子表；圆 = 原子。
C 的直接元素只有 2 个：原子 a 和子表 (b,c,d)，
所以 C 的长度是 2，不是 4。

图 3.1　用“框套框”看广义表：一对括号就是一个框

正例：$(a,(b,c))$ 是广义表，元素有原子也有子表；(x,y,z) 也是广义表——所有元素都是原子时，它就是一个普通的线性表。

反例：a 本身不是广义表，它是原子；$(a,b$ 少了右括号，不是合法的表示。

3.2　表头、表尾与长度

❑ 表头与表尾（课件第 60 页 / 笔记第 8 页）

当广义表 LS **非空**时，称第一个数据元素 a_1 为 LS 的**表头**(Head)，记作 $\text{head}(LS)=a_1$；称 LS 中**除去表头后其余数据元素组成的广义表**为 LS 的**表尾**(Tail)，记作 $\text{tail}(LS)=(a_2,\ldots,a_n)$。广义表 LS 中直接数据元素的个数称为 LS 的**长度**。

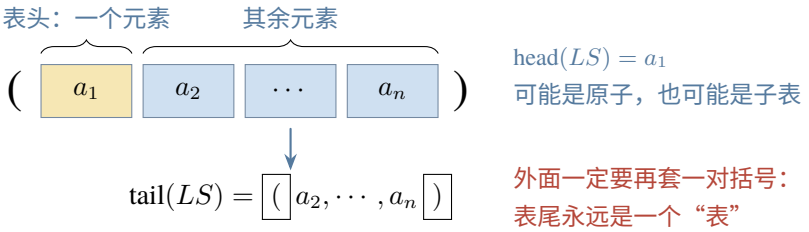


图 3.2　表头取“第一个元素本身”，表尾取“其余元素组成的表”

✘ 易错点：表尾不是最后一个元素（笔记红字）

错误想法：(a,b,c) 的表尾是 c。　**错在：**把“尾”理解成“最后一个”。

正确想法：表尾 = 去掉表头后剩下的所有元素组成的表，$\text{tail}((a,b,c))=(b,c)$。

两个推论：① 只有一个元素的表，表尾是空表：$\text{tail}((a))=()$；② 表头的“括号层数”保持原样：$\text{head}(((a,b),c))=(a,b)$，不是 a。

空表没有表头和表尾：$\text{head}(())$、$\text{tail}(())$ 无定义。

来源	广义表	长度	表头	表尾	说明
课件	$A=()$	0	无	无	空表
课件	$B=(e)$	1	e	$()$	只有一个原子
课件	$C=(a,(b,c,d))$	2	a	$((b,c,d))$	原子 + 子表
课件	$D=(A,B,C)$	3	$A=()$	(B,C)	代入后 $D=((),(e),(a,(b,c,d)))$
课件	$E=(a,E)$	2	a	(E)	递归表，$E=(a,(a,(a,\cdots)))$
笔记	$B=(())$	1	$()$	$()$	唯一元素是空表
笔记	$C=(a,(b,c))$	2	a	$((b,c))$	
笔记	$D=(x,y,z)$	3	x	(y,z)	就是线性表
笔记	$E=(C,D)$	2	C	(D)	两个元素都是子表
笔记	$F=(a,F)$	2	a	(F)	递归表

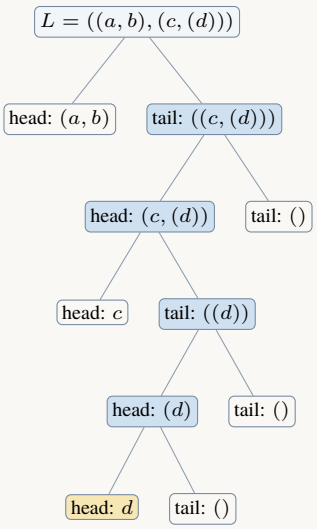
表 3.1　课件与笔记中的全部例子（笔记里 $A,B,\ldots$ 与课件同名但内容不同，分开看）

为什么 / 直观理解：() 和 (()) 有什么不同

() 是空表，长度 0，里面什么也没有；(()) 是“装着一个空表的表”，长度 1。就像一个空盒子和“装着一个空盒子的盒子”——后者打开后还能拿出一样东西。

例 3.1　用 head / tail 取出指定原子

已知 $L = ((a, b), (c, (d)))$，用 head 与 tail 的组合取出原子 d。



思路： d 在第 2 个元素里，先用 tail 跳过第 1 个元素，再用 head 把第 2 个元素“拿出来”；每进一层都重复“定位 → 取出”。

逐步： $\text{tail}(L) = ((c, (d))) \to \text{head}(\cdot) = (c, (d)) \to \text{tail}(\cdot) = ((d)) \to \text{head}(\cdot) = (d) \to \text{head}(\cdot) = d$。

结果： $\boxed{d = \text{head}(\text{head}(\text{tail}(\text{head}(\text{tail}(L)))))}$——写的时候从里往外读，最里层先做。

检查： 最后一步 head((d)) 得到的是 (d) 而不是 d，所以还需要再取一次 head——这是最容易漏掉的一步。

★ 生活中的它：排队买奶茶

生活场景： 奶茶店门口排着一队人，店员只能先服务队首的那个人。

对应关系： 整个队伍 = 广义表；队首那个人 = 表头（是“一个人”）；剩下的人 = 表尾（仍然是“一个队伍”，哪怕只剩一个人，甚至没有人）。如果队伍里有一个“三人小团体”一起点单，这个小团体就是子表。

小判断： 队伍 $(\text{甲}, (\text{乙}, \text{丙}), \text{丁})$ 的长度是 3（小团体算一个单位）；服务完甲之后，剩下的队伍是 $((\text{乙}, \text{丙}), \text{丁})$。

边界说明： 真实排队中人会插队、离队；广义表的“取表头 / 表尾”只是不断拆分，并不改变原表。

3.3　广义表的深度

❑ 深度（课件第 62 页 / 笔记第 8–9 页）

广义表 LS 中括号的**最大嵌套层数**称为 LS 的**深度**（笔记：广义表展开后所含括号的重数）。形式化定义为

$$\mathrm{Depth}(GL)=\begin{cases}0, & GL \text{ 为原子元素}\\ 1, & GL \text{ 为空表}\\ 1+\max\{\mathrm{Depth}(a_i)\mid 1\le i\le n\}, & \text{其他情况}\end{cases}$$

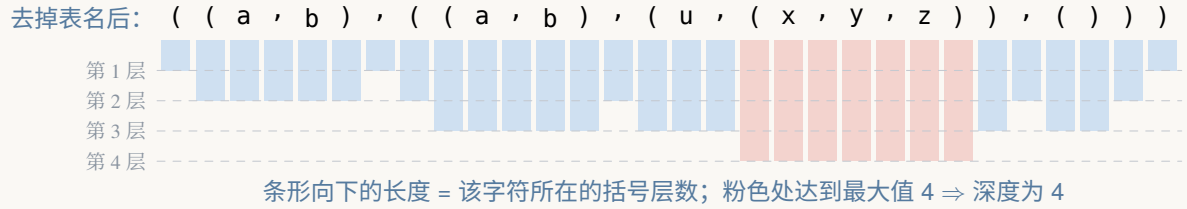
公式的意思：原子没有括号，深度 0；空表只有自己一对括号，深度 1；一般的表在“最深的那个元素”外面再套一层括号，所以是 $1+\max$。这是一个**递归定义**——表的深度由它的元素的深度决定。

课件的例子：2 深度为 0（括号层数为 0）；$()$ 深度为 1；$(2,(3,6))$ 深度为 2。**笔记的例子：**$A=(b,c)\to 1$，$B=(A,a)\to 2$，$C=(f,B,h)\to 3$。

例 3.2（课件第 75 页）　两种方法求深度

求 $E(B(a,b),\ D(B(a,b),\ C(u,(x,y,z)),\ A()))$ 的深度。（写法 $B(a,b)$ 表示名为 B 的表 (a,b)。）

$$E\,(\,B\,(a,b),\ D\,(\,B\,(a,b),\ C\,(u,\ (x,y,z)\,),\ A\,(\,)\,)\,)$$



条形向下的长度 = 该字符所在的括号层数；粉色处达到最大值 4 ⇒ 深度为 4

方法一（递归定义，自内向外）：$B=(a,b)$：元素都是原子，$1+0=1$；(x,y,z)：1；$C=(u,(x,y,z))$：$1+\max(0,1)=2$；$A=()$：空表，1；$D=(B,C,A)$：$1+\max(1,2,1)=3$；$E=(B,D)$：$1+\max(1,3)=\boxed{4}$。

方法二（数括号，见上图）：把表名去掉，写成 $((a,b),((a,b),(u,(x,y,z)),()))$，从左往右扫描：遇“(”层数 $+1$，遇“)”层数 -1，图中每个字符下方的条形高度就是它所在的层数，最大值就是深度——在 (x,y,z) 处层数达到 4。

检查：两种方法结果相同。✓

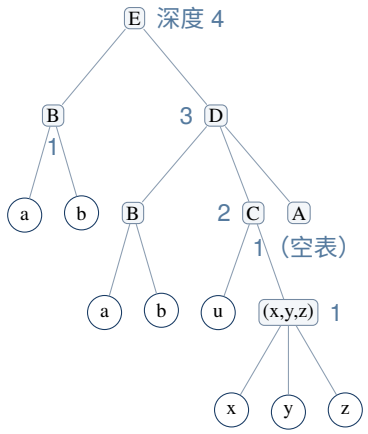


图 3.3　把广义表画成“树”：方框是表（子表），圆是原子；某个表的深度 = 从它出发向下经过的方框的最多个数

✘ 易错点

错误 1：把长度和深度混淆。长度只数最外层有几个元素；深度看最深嵌套了几层。$((a,b,c,d,e))$ 长度为 1、深度为 2。

错误 2：空表深度写成 0。空表本身有一对括号，深度为 **1**；原子才是 0。

错误 3：递归表求深度。$E=(a,E)$ 展开为 $(a,(a,(a,\cdots)))$，括号无限嵌套，深度为 ∞（笔记第 9 页），长度却是 2。

C++ 小程序（帮助理解，非考试要求的存储结构）：课件中广义表的存储结构（头结点、原子结点、表结点）注明不讲，下面只用字符串表示广义表，按定义实现 head、tail、长度和递归求深度，用来核对本章的例子：

C++　08_glist.cpp（节选）：按定义求表头、表尾、长度、深度

```cpp
// 把 "(e1,e2,...)" 拆成最外层的各个元素（只在括号层数为 1 的逗号处切开）
vector<string> Split(const string &ls)
{
    vector<string> items;
    string cur;
    int level = 0;
    for (size_t p = 1; p + 1 < ls.size(); p++) {   // 去掉最外层的一对括号
        char ch = ls[p];
        if (ch == ',' && level == 0) { items.push_back(cur); cur = ""; continue; }
        if (ch == '(') level++;
        if (ch == ')') level--;
        cur += ch;
    }
    if (!cur.empty()) items.push_back(cur);
    return items;
}
int Length(const string &ls) { return (int)Split(ls).size(); }
string Head(const string &ls) { return Split(ls)[0]; }      // 表头：第一个元素
string Tail(const string &ls)                                // 表尾：其余元素组成的“表”
{
    vector<string> it = Split(ls);
    string t = "(";
    for (size_t k = 1; k < it.size(); k++) t += (k > 1 ? "," : "") + it[k];
    return t + ")";                                          // 一定带括号
```

```
}
int Depth(const string &ls)                                    // 按递归定义求深度
{
    if (ls[0] != '(') return 0;                                // 原子
    vector<string> it = Split(ls);
    if (it.empty()) return 1;                                  // 空表
    int mx = 0;
    for (size_t k = 0; k < it.size(); k++) {
        int d = Depth(it[k]);
        if (d > mx) mx = d;
    }
    return 1 + mx;                                             // 1 + 子元素最大深度
}
```

运行输出（g++ 实测）

```
((a,b),(c,d))  head=(a,b)  tail=((c,d))  len=2  depth=2
(a,(b),())  head=a  tail=((b),())  len=3  depth=2
(a,(b,c),((d)))  head=a  tail=((b,c),((d)))  len=3  depth=3
((),(e),(a,(b,c,d)))  head=()  tail=((e),(a,(b,c,d)))  len=3  depth=3
(())  head=()  tail=()  len=1  depth=2
(a,((),(())),(b))  head=a  tail=(((),(())),(b))  len=3  depth=4
E depth = 4
d
```

读代码： `Split` 只在“括号层数为 0”（即最外层）的逗号处切分，所以子表不会被拆开——这正体现了“长度只数最外层”。`Tail` 返回时一定补上括号，对应“表尾是表”。`Depth` 的三个 `return` 与递归定义的三种情况一一对应。（只处理单字符原子，演示用。）

➢ 对照：广义表的深度 ↔ 第 3 章“栈”的括号匹配（思想重现型）

一句话定位： 用栈检查括号是否匹配时，扫描过程中栈里“(”的最大个数，就是广义表的深度（例 3.2 的方法二）。

栈 / 括号匹配中的结论	迁移	理由
遇左括号进栈、右括号出栈	✓	即“层数 $+1$ / -1”
栈的最大高度 = 深度	△	要求表达式里没有表名，或表名不带额外括号；空表 () 时栈高 1，与“空表深度 1”一致
括号匹配能判断“递归表”的深度	✗	递归表写不成有限长的括号串，深度 ∞

3.4　广义表的性质

❑ 广义表的性质（课件第 63 页，结合笔记第 8–9 页）

a) 广义线性： 对任意广义表，若不考虑其数据元素的内部结构，则它是一个线性表，直接数据元素之间是线性关系（笔记：元素有相对次序，一个直接前驱和一个直接后继）。

b) 数据元素复合性： 数据元素分为单数据元素（原子）和子表两种，因此广义表中数据元素的类型不统一。

c) 数据元素递归性：广义表可以是一个递归表，即自己可以作为自己的子表，如 $E = (a, E)$。
d) 数据元素共享性：广义表以及广义表的数据元素可以被其他广义表共享，如 $A = ()$，$B = (A) = (())$。
笔记补充：广义表是**多层次结构**，可以用**树 / 有向图**表示。

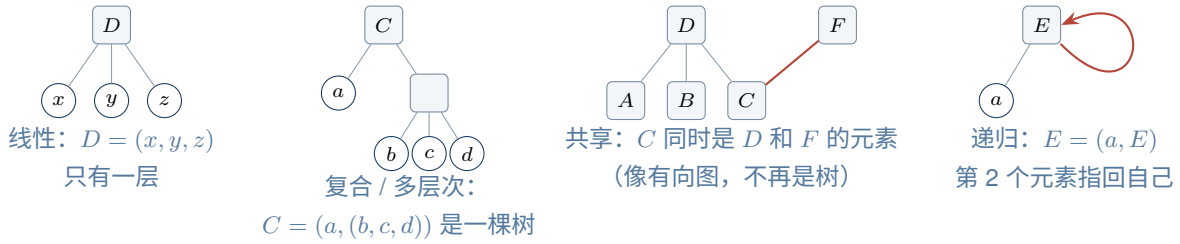


图 3.4　广义表的四种形态：线性 ⊂ 多层次（树）⊂ 共享（有向图）⊂ 递归（含回路）

★ 生活中的它：电脑里的文件夹

生活场景：电脑“文档”文件夹里有文件，也有子文件夹，子文件夹里又有文件和文件夹。
对应关系：文件夹 = 广义表；文件 = 原子；子文件夹 = 子表；“打开文件夹最多要点几层” = 深度；“文件夹第一层有几项” = 长度；Windows 的快捷方式让同一个文件夹出现在两处 = **共享**；在文件夹里放一个指向它自己的快捷方式 = **递归**。
小判断：“课程 / [讲义.pdf, 代码 / [a.cpp, b.cpp], 空文件夹 /]”写成广义表就是 $(p, (a, b), ())$：长度 3，深度 2。
边界说明：真实文件系统中文件夹内的项通常按名称排序，而广义表的元素次序由定义给定。

3.5　广义表的抽象数据类型与基本操作

❑ 广义表的 ADT（课件第 64 页，已改正一处标签）

ADT GList {
数据对象：$D = \{e_i \mid i = 1, 2, \ldots, n;\ n \geq 0;\ e_i \in \text{AtomSet}$ 或 $e_i \in \text{GList}\}$，AtomSet 为某个数据对象
数据关系：$R = \{\langle e_{i-1}, e_i \rangle \mid e_{i-1}, e_i \in D,\ 2 \leq i \leq n\}$
基本操作：……
} ADT GList

原图纠错 7：课件第 64 页 ADT 的第二个标签

原始写法：第二行也写作“数据对象：$R = \{\ldots\}$”。　**问题：**R 描述的是元素之间的关系。　**规范写法：“数据关系：$R = \{\langle e_{i-1}, e_i \rangle \mid \ldots\}$”。**它说明广义表的直接元素之间是线性关系——与性质 a) 一致。

基本操作（课件第 65–66 页，笔记第 9 页）：

操作	操作结果（初始条件：广义表已存在）	例：对 $L=(b,(c))$
`First()` / GetHead	返回广义表的第一个元素	第一个元素 b
`Next(elemPtr)`	返回 `elemPtr` 指向的广义表元素的后继	b 的后继是 (c)
GetTail	求表尾	$((c))$
`Empty()`	如广义表为空，返回 `true`，否则返回 `false`	`false`
`Push(const ElemType &e)`	将原子 e 作为表头加入到广义表最前面	`Push(a)` 后：$(a,b,(c))$
`Push(GenList &subList)`	将子表 `subList` 作为表头加入到广义表最前面	`Push`$((x))$ 后：$((x),b,(c))$
`Depth()`	返回广义表的深度	2

注意两个 Push 的区别：压入原子 a，长度加 1、深度不变；压入子表 (x)，长度也只加 1（子表算一个元素），但新的表头是一个表。C++ 中这两个同名函数靠**参数类型不同**实现**函数重载**。

了解即可（课件第 67–77 页注明“不讲”）

课件中的“引用数法广义表”把结点分三类：头结点（`tag=HEAD`，存引用数 `ref`）、原子结点（`tag=ATOM`，存原子值）、表结点（`tag=LIST`，`subLink` 指向子表的头结点），三类结点都有 `nextLink` 指向后继。求深度的 `DepthHelp` 正是按递归定义写的：空表返回 1，否则对每个 `tag==LIST` 的元素递归求深度，取最大值再加 1——与上面 `08_glist.cpp` 的 `Depth` 思路相同。

➢ 对照：广义表 ↔ 线性表（推广型）

一句话定位：线性表是“元素全为原子”的广义表；广义表放开了“元素必须是单个数据”这一限制。

比较维度	线性表 $(a_1,\ldots,a_n)$	广义表 $(a_1,\ldots,a_n)$
元素	同一类型的单个数据	原子或子表，类型不统一
长度	元素个数	最外层元素个数
层次	一层	多层（有深度）
“尾”的含义	常指最后一个元素（尾结点）	表尾 = 除表头外其余元素组成的**表**

线性表中的结论	迁移	理由或反例
直接元素之间是线性关系	✓	性质 a) 广义线性
长度 = 包含的数据个数	✗	$(a,(b,c,d))$ 含 4 个原子，长度却是 2
“尾”是最后一个元素	✗	广义表的表尾是表：$\text{tail}((a,b))=(b)$
可以顺序存储，按下标随机存取	△	元素大小不一（原子或整张子表），课件采用链式存储

> **同一个小算例：**(x, y, z) 当作线性表：长度 3，尾结点是 z；当作广义表：长度 3，表尾是 (y, z)，深度 1。

本章知识地图与小结

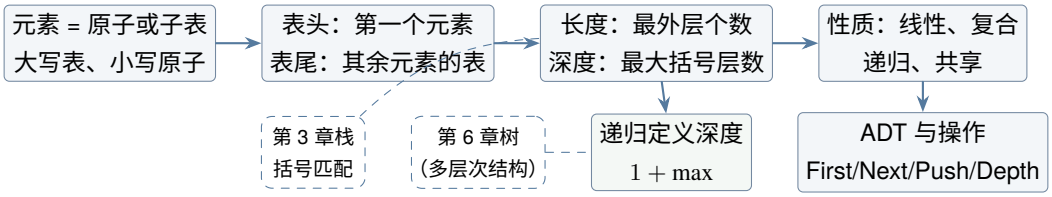


图 3.5　第 3 章知识地图

✓ 到这里，我们已经解决了什么？

广义表把线性表的元素推广为“原子或子表”。表头是第一个元素本身，表尾是其余元素组成的表；长度数最外层，深度数最大嵌套层数（原子 0，空表 1，其余 $1 + \max$）。广义表直接元素之间仍是线性的，但整体是多层次结构，可以共享、可以递归。

下一步：笔记最后一页写道“广义表是多层次结构 $\rightarrow$ 树 / 有向图”——这正是第 6 章“树与二叉树”的入口：不共享、不递归的广义表就是一棵树。

学完这里，你应该能够回答

1. 广义表与线性表的根本区别是什么？
2. (a, b, c) 的表尾是什么？$((a))$ 的表头是什么？(a) 的表尾是什么？
3. $()$ 与 $(())$ 的长度、深度各是多少？
4. 写出深度的递归定义，并用它求 $(a, (b, (c)), ())$ 的深度。
5. 递归表 $E = (a, E)$ 的长度和深度各是多少？
6. 两个 `Push` 操作有何不同？C++ 用什么机制让它们同名？

知识点—题目对应表

核心知识点	基础巩固（1–25）	提升 / 跨学科（26–38）	涉及的对照
数组定义、特点、基本操作、逻辑结构	1	—	数组 ↔ 顺序表
行序 / 列序存储	4, 11	26, 36	C++ ↔ MATLAB
一维、二维地址公式	3, 16, 17	26, 36	—
一般下标地址公式	2（课件题）	31	—
三维 / n 维地址、c_i 递推	21（作业 5.1）	32	进位计数制
C++ 数组声明、`sizeof`	11	—	—
特殊矩阵与稀疏矩阵的区别	12	—	—
对称矩阵压缩	6, 15, 18	35, 37	线性代数对称矩阵
三角矩阵压缩	5（课件题）, 15, 24	33	线性代数三角矩阵
三对角矩阵压缩（两种映射）	10, 15	27, 37	弹簧链 / 电阻链
稀疏矩阵、稀疏因子	20	38	邻接矩阵
三元组顺序表（建立、还原、特点）	13, 22（课件题）, 23	34, 38	—
`SetElem` / `SimpleTranspose` / 乘法	—	29, 30, 34	选课表转置
带行指针的链表	7	—	图的邻接表
广义表表头、表尾	8（课件题）, 14, 25	—	线性表的“尾”
广义表长度、深度、性质、基本操作	9, 14, 19, 25	28	括号匹配（栈）

正式练习

共 38 题：A. 基础巩固 25 题　B. 中等偏下提升 10 题　C. 跨学科迁移 3 题。题中凡有图，图中只画题目已给出的信息。答案与解析集中在练习之后，请先独立完成。

A 基础巩固训练（25 题）

一、单项选择题（每题只有一个正确选项）

1. 下列关于数组的说法中，正确的是（　）。

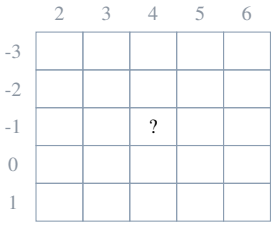
A. 数组中各元素的数据类型可以不同

B. 数组一旦定义，它的维数和维界就不再改变

C. 数组的基本操作包括插入元素和删除元素

D. 二维数组中每个元素最多只有一个直接前驱

2.（课件第 56 页）有一矩阵为 $A[-3:1,\ 2:6]$，每个元素占一个存储单元，存储的首地址为 100，以行序为主，则元素 $a_{-1,4}$ 的地址为（　）。

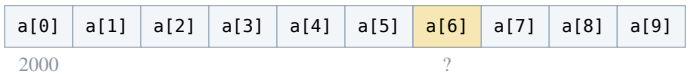


行下标 $-3 \sim 1$，列下标 $2 \sim 6$，“?”为 $a_{-1,4}$

A. 111　　B. 112

C. 113　　D. 125

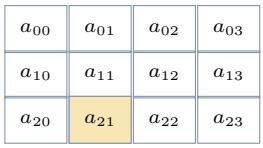
3. `int a[10];` 每个元素占 4 字节，`a[0]` 的地址为 2000，则 `a[6]` 的地址为（　）。



A. 2006　　B. 2020

C. 2024　　D. 2028

4. 二维数组 `int a[3][4]` 若**按列序为主序**存放，以 `a[0][0]` 为第 0 号单元，每个元素占 1 个单元，则 `a[2][1]` 存放在第（　）号单元。



3 行 4 列，按**列**存放

A. 6　　B. 9

C. 5　　D. 7

5.（课件第 57 页）一个 $n \times n$ 的上三角矩阵 $A = [a_{ij}]$，将其中元素 a_{ij} $(i \le j)$ 按行序为主序顺序存储在一维数组 $B[1..n(n+1)/2]$ 中，则 a_{ij} 在 B 中的位置是（　）。

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22} & \cdots & a_{2n} \\ & & \ddots & \vdots \\ & & & a_{nn} \end{pmatrix}$$

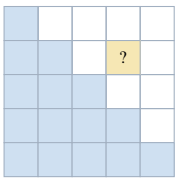
A. $(i-1)(2n+i)/2+i-j+1$

B. $(i-1)(2n-i+2)/2+j-i+1$

C. $(i-1)(2n-i)/2+j-i$

D. $(i-1)(2n-i+2)/2+j-i$

6. 5 阶对称矩阵只存下三角（含对角线），按行序存入 `sa[0..14]`（i, j 从 1 开始），则 $a_{2,4}$ 存放在（　）。



蓝色 = 存储的下三角；“?” = $a_{2,4}$（未存储）

A. `sa[4]`　　B. `sa[7]`

C. `sa[8]`　　D. `sa[10]`

7. 与三元组顺序表相比，稀疏矩阵的“带行指针的链表”的主要优点是（　）。

A. 存储密度更高，不需要存储指针

B. 可以由第 i 个行指针直接进入第 i 行，不必从表头开始查找

C. 能在 $O(1)$ 时间内按 (i, j) 随机存取任意元素

D. 结点中不需要存储列号

8.（课件第 78 页）广义表 $((a, b), (c, d))$ 的表尾是（　）。

A. (c, d)　　B. $((c, d))$

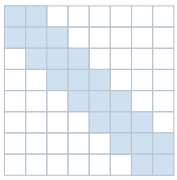
C. (d)　　D. d

9. 广义表 $(a, (b, c), ((d)))$ 的长度和深度分别是（　）。

A. 3，3　　B. 3，2

C. 4，3　　D. 5，3

10. 100 阶三对角矩阵按行压缩，只存放 $|i - j| \le 1$ 的元素（不另存常数），共需（　）个存储单元。



示意（画出 8 阶）：蓝色为 $|i - j| \le 1$ 的带状区域

A. 300　　B. 298

C. 297　　D. 5050

二、多项选择题（每题有两个或两个以上正确选项）

11. 关于 C++ 中数组的存储，下列说法正确的是（　）。（int 占 4 字节）

A. C++ 的二维数组按行序为主序存放

B. 按行序存放时，`a[i][j]` 与 `a[i][j+1]` 在内存中相邻（$j + 1$ 不越界）

C. 按列序存放时，`a[i][j]` 与 `a[i][j+1]` 在内存中相邻

D. 定义 `int a[3][4];` 后，`sizeof(a)` 的值为 48

12. 下列矩阵中，属于课件所说的“特殊矩阵”、能用公式 $k = f(i, j)$ 直接算出压缩下标的有（　）。

A. 对称矩阵　　B. 下三角矩阵

C. 三对角矩阵　　D. 非零元随机分布的稀疏矩阵

13. 关于稀疏矩阵的三元组顺序表，下列说法正确的是（　）。

A. 每个非零元用（行号，列号，元素值）表示

B. 为了保持规整，可以存入值为 0 的元素占位

C. 三元组按行号递增排列，行号相同时按列号递增排列

D. 可以按 (i, j) 随机存取任意元素

14. 对广义表 $L = (a, (b), ())$，下列说法正确的是（　）。

A. L 的长度为 3

B. L 的深度为 2

C. $\text{head}(L) = (a)$

D. L 的第 3 个元素 $()$ 的深度为 1

15. n 阶矩阵压缩存储时所需的存储单元数（不计行列数等额外信息），下列说法正确的是（　）。

A. 对称矩阵：$n(n+1)/2$

B. 下三角矩阵（右上方为同一常数 c）：$n(n+1)/2 + 1$

C. 三对角矩阵（不另存常数）：$3n$

D. 对称矩阵存上三角比存下三角节省空间

三、填空题

16. （笔记第 3 页例题）`int a[5];` 每个元素占 4 字节，`a[0]` 存储在 2000 单元，则 `a[4]` 占用的地址范围是 _____ ～ _____。

17. 二维数组 `a[4][5]` 按行序存放，$\text{Loc}(0, 0) = 1000$，每个元素占 2 个单元，则 `a[2][3]` 的地址为 _____。

18. 10 阶对称矩阵压缩存储（只存下三角含对角线）共需 _____ 个单元；按行序存入从 0 开始的 `sa` 时，$a_{5,3}$ 的下标为 _____。

19. (1) 递归表 $E = (a, E)$ 的长度为 _____，深度为 _____。
(2) 对广义表 $L = (b, (c))$ 先执行 `Push`(原子 a), 再执行 `Push`(子表 (x)), 则 $L =$ _____，此时长度为 _____。

20. 课件第 31 页的稀疏矩阵 $A_{6\times 7}$ 共有 ____ 个非零元，稀疏因子 $\delta =$ ____（保留两位小数）。

$$A_{6\times 7}=\begin{pmatrix}0&0&0&22&0&0&15\\0&11&0&0&0&17&0\\0&0&0&-6&0&0&0\\0&0&0&0&0&39&0\\91&0&0&0&0&0&0\\0&0&28&0&0&0&0\end{pmatrix}$$

四、解答题

21.（课件第 79 页作业 5.1）有三维数组 $A[0:7,\ 0:8,\ 0:9]$，数组的起始地址是 1000，每个元素长为 2，按行序为主序存放。试给出：(1) 元素 $a_{1,6,8}$ 的起始地址；(2) 数组 A 所占用的存储空间。

22.（课件第 58 页 / 作业 5.2）已知稀疏矩阵如下所示，试写出三元组表示（含总体信息）。

$$\begin{pmatrix}15&0&0&0&16\\0&6&0&0&0\\0&0&8&0&0\\9&0&0&18&0\end{pmatrix}$$

23. 已知某稀疏矩阵三元组表的总体信息为 $(4,5,5)$，三元组依次为 $(1,2,7),(2,5,-1),(3,1,4),(3,3,6),(4,4,2)$。(1) 还原出该矩阵；(2) 这个三元组表是否按行序有序？说明理由。

24. 4 阶下三角矩阵（右上方常数 $c=0$）如下，按课件约定压缩：`sa[0]` 存常数，下三角元素从 `sa[1]` 起按行存放。(1) 写出 `sa`；(2) 求 $a_{3,2}$、$a_{2,4}$ 的下标和值；(3) 共需多少个单元？

$$\begin{pmatrix}9&0&0&0\\5&3&0&0\\6&1&2&0\\8&4&3&4\end{pmatrix}$$

25. 已知广义表 $L=(a,((b,c),d),(e))$。(1) 求 head(L) 与 tail(L)；(2) 求 L 的长度和深度；(3) 用 head 和 tail 的组合取出原子 c。

B 中等偏下提升训练（10 题）

一、单项选择题

26. 二维数组 $A[0:3,\ 0:4]$（4 行 5 列），按行序存放时元素 $a[2][3]$ 的存储位置，与按列序存放时（基址、元素长度都相同）哪个元素的存储位置相同？（　）

a_{00}	a_{01}	a_{02}	a_{03}	a_{04}
a_{10}	a_{11}	a_{12}	a_{13}	a_{14}
a_{20}	a_{21}	a_{22}	a_{23}	a_{24}
a_{30}	a_{31}	a_{32}	a_{33}	a_{34}

已知：按**行**存放时的 $a[2][3]$

A. $a[3][2]$　　B. $a[1][3]$

C. $a[3][1]$　　D. $a[2][3]$

27. 阅读课程代码 **三对角矩阵.cpp** 中的取元素运算符。若定义 `TriDiagonalMatrix<int> a(5);`，则 `a(4,3)` 返回的是哪个单元的引用？（　）

C++　三对角矩阵.cpp（节选）

```cpp
if (row - col == 1) return elems[2 * order + row - 2];     //元素在低对角线上
else if (row - col == 0) return elems[order + row - 1];    //元素在主对角线上
else if (row - col == -1) return elems[row];               //元素在高对角线上
else return elems[0];                                      //元素在其他位置
```

A. `elems[9]`　　B. `elems[12]`

C. `elems[11]`　　D. `elems[3]`

28. 设 $A=()$，$B=(A)$，$C=(A,B)$，$D=(a,C,(b))$，则 D 的长度和深度分别为（　）。

A. 3，4　　B. 3，3

C. 5，4　　D. 4，3

二、多项选择题

29. 对 m 行 n 列、含 t 个非零元的稀疏矩阵执行课程代码中的 `SimpleTranspose`，下列说法正确的是（　）。

A. 时间复杂度为 $O(n\cdot t)$

B. 转置后的三元组表 `dest` 仍按行序有序

C. 当 t 接近 $m\cdot n$ 时，它比直接转置二维数组更快

D. 外层循环按 `source` 的行号 $1\sim m$ 进行

30. 某 5 行 6 列稀疏矩阵的三元组表（`num=3`，`maxSize` 足够大）当前为：下标 0：$(1,3,2)$；下标 1：$(2,6,8)$；下标 2：$(3,1,1)$。对它分别执行课程代码的 `SetElem`，下列说法正确的是（　）。

A. 执行 `SetElem(2,2,5)` 后，新三元组位于下标 1，原来的 $(2,6,8)$、$(3,1,1)$ 各后移一位

B. 执行 `SetElem(2,6,0)` 后，$(2,6,8)$ 被删除，`num` 变为 2

C. 执行 `SetElem(4,4,0)` 后，表中插入一个值为 0 的三元组

D. 执行 `SetElem(7,1,3)` 返回 `false`

三、填空题

31. 二维数组 $a[-2:3,\ 1:4]$，每个元素占 4 个单元，$\mathrm{Loc}(-2,1)=500$。按列序为主序存放时 $a[1][3]$ 的地址为 ____；按行序为主序存放时为 ____。

32. `int a[4][5][6];`（int 占 4 字节）按 n 维公式 $\mathrm{Loc}=\mathrm{Loc}(0,0,0)+\sum c_i j_i$，则 $c_3=$ ____，$c_2=$ ____，$c_1=$ ____；`a[2][3][4]` 相对 `a[0][0][0]` 的偏移为 ____ 字节。

33. 5 阶上三角矩阵按笔记约定存储：$a_{ij}\ (i\le j)$ 按行序存入 $B[1..15]$，下方常数 c 存于 $B[16]$。则 $a_{4,5}$ 在 $B[$____$]$，$a_{4,1}$ 在 $B[$____$]$。

四、综合大题

34.（稀疏矩阵三元组的完整流程，数据取自课件第 46 页）已知

$$A=\begin{pmatrix}3&0&0&7\\0&0&0&-1\\0&2&0&0\end{pmatrix},\qquad B=\begin{pmatrix}0&1\\2&0\\3&4\\0&0\end{pmatrix}.$$

(1) 写出 A 的三元组表（含总体信息）；
(2) 用 `SimpleTranspose` 求 A^{T} 的三元组表，写出外层循环每一轮找到的三元组；
(3) 求 $C=A\times B$，并写出 C 的三元组表；
(4) c_{11} 的计算过程中出现了非零乘积吗？为什么 C 的三元组表里没有 c_{11}？

35.（对称矩阵：公式 + C++）对称矩阵

$$S=\begin{pmatrix}2&1&0&5\\1&3&4&0\\0&4&6&7\\5&0&7&8\end{pmatrix}$$

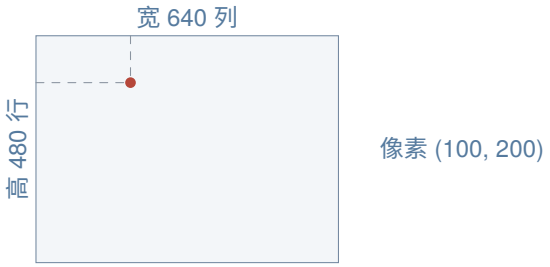
只存下三角，按行序存入从 0 开始的 `sa`。
(1) 写出 `sa`；
(2) 求 $a_{2,3}$ 和 $a_{4,1}$ 的下标与值；
(3) 补全下面 C++ 函数中的两处空白，使它返回 a_{ij} 在 `sa` 中的下标（i,j 从 1 开始）；
(4) 若 $n=100$、元素为 int（4 字节），压缩存储比常规存储节省多少字节？

C++　待补全

```cpp
int SymIndex(int i, int j)
{
    if (i >= j) return ____(1)____;
    else        return ____(2)____;
}
```

C 跨学科迁移训练（3 题）

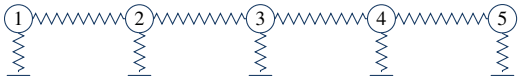
36.（数字图像处理）一张灰度照片高 480 像素、宽 640 像素，每个像素用 1 字节的灰度值表示，程序中存放在 `unsigned char img[480][640];` 中（C++ 按行存放），`img[0][0]` 的偏移为 0。像素“第 100 行、第 200 列”（都从 0 数）即 `img[100][200]` 相对首地址的偏移是（　）字节。



A. 64200　　B. 96100

C. 64000　　D. 128400

37.（电路）由 n 个节点串成一条链的电阻网络，每个节点只通过电阻与相邻节点（及参考地）相连。题目直接告诉你：该网络节点方程的系数矩阵 G（n 阶）中，只有主对角线和紧邻它的上、下两条对角线上的元素可能非零，并且 $g_{ij} = g_{ji}$。下列说法正确的是（　）。



节点 i 只与 $i \pm 1$ 及地相连（图为 $n = 5$）

A. G 是三对角矩阵

B. G 是对称矩阵，可以只存下三角的 $n(n+1)/2$ 个元素

C. 同时利用“三对角”和“对称”，只需存主对角线和一条次对角线，共 $2n - 1$ 个元素

D. $n = 1000$ 时，按三对角矩阵按行压缩（不另存常数）需要 3000 个单元

38.（网络科学）某校园 App 有 2000 名用户，“关注”关系用 2000×2000 的矩阵表示：用户 i 关注用户 j 则 $a_{ij}=1$，否则为 0；共有 12000 条关注关系。每个整数占 4 字节。
(1) 用三元组表（含 1 个总体信息三元组，每个三元组 3 个整数）存储，共需 ____ 字节；
(2) 用常规二维 int 数组存储需要 ____ 字节；
(3) 稀疏因子 $\delta=$ ____。

答案速查

题组	答案
基础单选 1–10	1. B　2. B　3. C　4. C　5. B　6. B　7. B　8. B　9. A　10. B
基础多选 11–15	11. ABD　12. ABC　13. AC　14. ABD　15. AB
基础填空 16–20	16. 2016, 2019　17. 1026　18. 55, 12　19. 2, ∞; $((x),a,b,(c))$, 4　20. 8, 0.19
基础解答 21–25	21. 1316；1440 个单元　22. $(4,5,6)$，$(1,1,15)$，$(1,5,16)$，$(2,2,6)$，$(3,3,8)$，$(4,1,9)$, $(4,4,18)$ 23. 见解析，是按行序有序　24. sa$=[0,9,5,3,6,1,2,8,4,3,4]$；$k=5$（值 1），$k=0$（值 0）；11 个 25. head $=a$，tail $=(((b,c),d),(e))$；长度 3，深度 3； $c=\text{head}(\text{tail}(\text{head}(\text{head}(\text{tail}(L)))))$
提升单选 26–28	26. B　27. B　28. A
提升多选 29–30	29. AB　30. ABD
提升填空 31–33	31. 560，556　32. 4，24，120，328　33. 14，16
综合 34–35	34. A：$(3,4,4)$, $(1,1,3)$, $(1,4,7)$, $(2,4,-1)$, $(3,2,2)$；A^{T}：$(4,3,4)$, $(1,1,3)$, $(2,3,2)$, $(4,1,7)$, $(4,2,-1)$；C：$(3,2,2)$, $(1,2,3)$, $(3,1,4)$ 35. sa$=[2,1,3,0,4,6,5,0,7,8]$；$a_{2,3}$：$k=4$，值 4；$a_{4,1}$：$k=6$，值 5；节省 19800 字节
跨学科 36–38	36. A　37. ABC　38. 144012；16000000；0.003

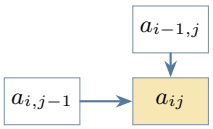
详细解析

A 基础巩固 · 单项选择题

第 1 题　答案：B　基础 · 概念

考点：数组的定义与特点（结构固定，只有初始化、销毁、取元素、改元素四种操作）；逻辑结构。

思路：逐项对照数组定义的三个关键词（相同类型、有限、按下标访问）和“结构固定”的特点。



选项 D 的反例：内部元素 a_{ij} 有 2 个直接前驱（上方、左方）

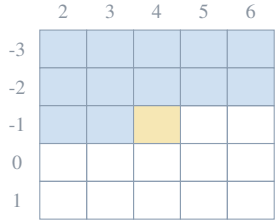
A 错：定义要求“相同类型”，这也是每个元素占相同单元数 L、地址公式成立的前提。**B 对：**这正是“结构固定”。**C 错：**数组只有四种基本操作，不能插入删除，否则会破坏维界。**D 错：**见上图，二维数组的内部元素有 2 个直接前驱、2 个直接后继，这正是它是非线性结构的原因。

常见错误：把顺序表的插入、删除“搬”到数组上（对照第 1.2 节迁移表中的 ✗）。

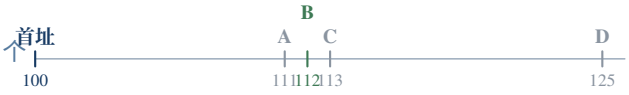
✓ **一句话总结：**数组 = 同类型 + 结构固定 + 只读改不增删。

第 2 题（课件第 56 页）　答案：B　基础 · 一般下标地址

考点：一般下标二维数组的行序地址公式 $\mathrm{Loc}(i,j)=\mathrm{Loc}(c_1,c_2)+[(d_2-c_2+1)(i-c_1)+(j-c_2)]L$。



前面：$-3,-2$ 两整行 $=10$ 个
本行在列 2、3 两个
共 12 个（蓝色）



四个选项在地址轴上的位置（B 恰好在“前面 12 个”处）

计算：列数 $d_2-c_2+1=6-2+1=5$；$i-c_1=-1-(-3)=2$；$j-c_2=4-2=2$。$\mathrm{Loc}=100+(5\times 2+2)\times 1=112$。

A（111）错：少数了 1 个，常见原因是把列数算成 $d_2-c_2=4$ 后又误算。**C（113）错：**多数了 1 个，常见原因是把 $j-c_2$ 写成 $j-c_2+1$。**D（125）错：**$125=100+25$，相当于把整个 5×5 数组都当作“前面的元素”。

常见错误：下标从负数开始时，“个数 $=d-c+1$”与“偏移 $=i-c$”混用。

✓ **一句话总结：**先平移成从 0 开始，再数“前面整行 + 本行前几个”。

第 3 题　答案：C　基础 · 一维地址

考点： $\mathrm{Loc}(i) = a + iL$。

计算： a[6] 前面有 6 个元素，$2000 + 6 \times 4 = 2024$。**A（2006）**忘了乘 L；**B（2020）**按 5 个元素算；**D（2028）**按 7 个元素算（把下标当成“第几个”）。

前面 6 个 × 4 字节 = 24

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
2000	2004	2008	2012	2016	2020	2024	2028

✓ **一句话总结：** C++ 下标 i 恰好等于前面元素的个数。

第 4 题　答案：C　基础 · 列序存储

考点： 列序公式 偏移 $= m \cdot j + i$（**乘行数**）。

列序：格中数字 = 单元号

0	3	6	9
1	4	7	10
2	5	8	11

对照：若按行序则是 9（选项 B）

0	1	2	3
4	5	6	7
8	9	10	11

计算： 列序时先存完第 0 列（3 个），再在第 1 列中数到第 2 个：$3 \times 1 + 2 = 5$。**A（6）**用了列数 4：$4 \times 1 + 2$；**B（9）**是行序结果；**D（7）**把行列号用反：$2 \times 3 + 1$。

✓ **一句话总结：** 行序乘列数，列序乘行数。

第 5 题（课件第 57 页）　答案：B　基础 · 上三角矩阵

考点： 上三角按行存：第 r 行有 $n - r + 1$ 个元素，本行从 a_{ii} 开始。

推导： 前 $i-1$ 行共 $n + (n-1) + \cdots + (n-i+2) = \frac{(i-1)(2n-i+2)}{2}$ 个；本行 a_{ij} 前有 $j - i$ 个；B 从 1 开始，位置再 $+1$：$\frac{(i-1)(2n-i+2)}{2} + j - i + 1$。

$n = 4$：格中数字 = 实际位置

1	2	3	4
	5	6	7
		8	9
			10

用 $a_{2,3}$（实际位置 6）检验四个公式：

A：$\frac{1\times10}{2} + 2 - 3 + 1 = 5$ ✗

B：$\frac{1\times8}{2} + 3 - 2 + 1 = 6$ ✓

C：$\frac{1\times6}{2} + 3 - 2 = 4$ ✗

D：$\frac{1\times8}{2} + 3 - 2 = 5$ ✗（少了 $+1$，是 0 起始的写法）

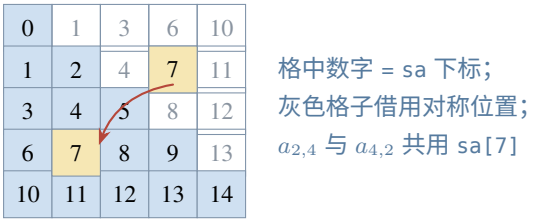
常见错误： 本行前写成 $j - 1$ 个（照搬下三角）；忘了 B 从 1 开始要 $+1$（选 D）。

✓ **一句话总结：** 选择题里的下标公式，代一个小例子检验最快最稳。

第 6 题　答案：B　基础 · 对称矩阵

考点： $i<j$ 时交换：$k=\frac{j(j-1)}{2}+i-1$。

计算： $i=2<j=4$，$k=\frac{4\times3}{2}+2-1=7$。**A（4）**没有交换，直接代 $\frac{2\times1}{2}+4-1$，取到的是 $a_{3,2}$；**C（8）**交换后忘了 -1；**D（10）**是 $a_{5,1}$ 的下标。



✓ **一句话总结：**上三角元素找它的“镜像”去查。

第 7 题　答案：B　基础 · 带行指针的链表

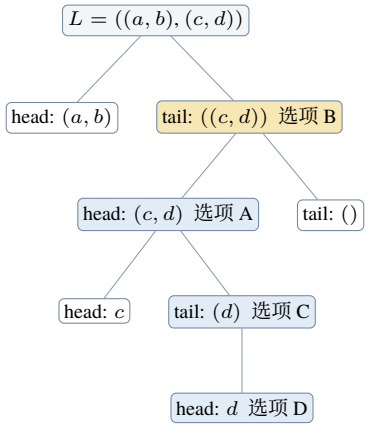
考点：带行指针的链表与三元组顺序表的比较。

分析：行指针数组的第 i 个指针直接指向第 i 行的链表（见图 2.16），所以找某一行不必从头扫描，B 对。**A 错：**链表结点要存指针，存储密度反而低于三元组顺序表。**C 错：**进入第 i 行后仍要沿链表顺序查找列号，不是 $O(1)$ 随机存取。**D 错：**同一行的结点要靠列号区分，必须存列号。

✓ **一句话总结：**行指针解决的是“快速找到某一行”，不是“随机存取任意元素”。

第 8 题（课件第 78 页）　答案：B　基础 · 表尾

考点：表尾 = 除表头外其余元素组成的**表**。

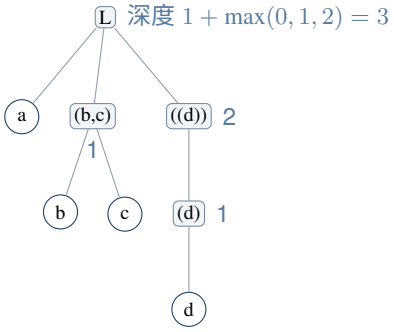


分析：L 有两个元素 (a,b) 和 (c,d)；表头是 (a,b)，其余元素只有 (c,d)，组成的表是 $((c,d))$。图中标出了四个选项各自是什么：A 是 head(tail(L))——“最后一个元素”，正是笔记红字警告的错误；C、D 是再往下拆的结果。

✓ **一句话总结：**表尾外面一定还有一层括号。

第 9 题　答案：A　基础 · 长度与深度

考点：长度 = 最外层元素个数；深度 = 1 + max（子元素深度）。



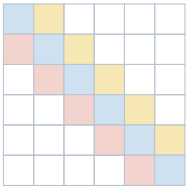
长度：最外层元素 a、(b,c)、$((d))$，共 3 个（排除 C、D：C 把 b,c 拆开数成 4；D 数了全部原子 5 个）。**深度：**$((d))$ 嵌套 2 层，加最外层共 3（排除 B）。

✓ **一句话总结：**长度看宽度，深度看高度。

第 10 题　答案：B　基础 · 三对角矩阵

考点：非零区共 $3n-2$ 个元素。

计算：主对角线 100 个 + 两条次对角线各 99 个 $=298$。**A（300）**把两条次对角线也算成 n 个；**C（297）**多减了 1；**D（5050）**是 $\frac{n(n+1)}{2}$，那是对称矩阵 / 三角矩阵的数量。



蓝：主对角线 n 个；黄、粉：次对角线各 $n-1$ 个

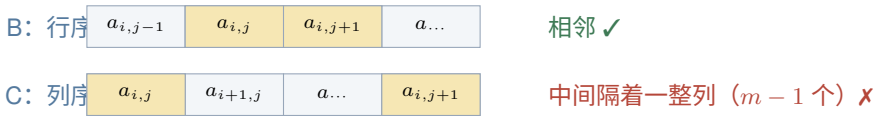
$n+2(n-1)=3n-2$；$n=100$ 时为 298

✓ **一句话总结：**三对角：$n+(n-1)+(n-1)$。

A 基础巩固 · 多项选择题

第 11 题　答案：ABD　基础 · 存储顺序与 C++

考点：行序 / 列序中“谁和谁相邻”；`sizeof`。



A 对：C/C++ 用行序（第 1.4 节程序实测）。**B 对、C 错：**见上图；列序时与 $a_{i,j}$ 相邻的是 $a_{i+1,j}$。**D 对：**$3\times4\times4=48$（第 1.1 节程序输出 48）。

✓ **一句话总结：**行序：同一行挨着；列序：同一列挨着。

第 12 题　答案：ABC　基础 · 特殊矩阵与稀疏矩阵

考点：特殊矩阵：非零元分布有规律 ⇒ 公式映射；稀疏矩阵：无规律 ⇒ 记录位置。



A、B、C 对：蓝色区域的形状固定，可由“数前面有几个”得出 $k = f(i,j)$。**D 错：**粉色非零元的位置没有规律，只能用三元组记录 (i,j)。

✓ **一句话总结：**有规律用公式，无规律记位置。

第 13 题　答案：AC　基础 · 三元组顺序表

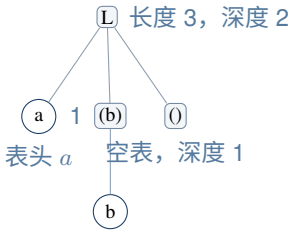
考点：三元组表的组成、有序性和缺点。

A 对：三元组 (i,j,a_{ij}) 唯一确定一个非零元。**B 错：**压缩原则是“零元素不分配空间”；课程代码的 `SetElem(r,c,0)` 甚至会把已有三元组删除。**C 对：**按行序有序，同行按列号递增（`SetElem` 插入时就是按这个顺序找位置的，见图 2.13）。**D 错：**这是三元组表的缺点：不能随机存取，`GetElem` 要顺序查找。

✓ **一句话总结：**三元组表：只存非零、按行有序、不能随机存取。

第 14 题　答案：ABD　基础 · 广义表

考点：长度、深度、表头；空表的深度。

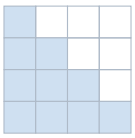


A 对：最外层 3 个元素。**B 对：**$1+\max(0,1,1)=2$。**C 错：**表头是元素 a 本身，不加括号。**D 对：**空表深度为 1。

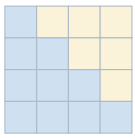
✓ **一句话总结：**表头不加括号，表尾才加括号。

第 15 题　答案：AB　基础 · 压缩存储单元数

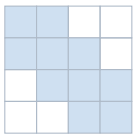
考点：对称 $\frac{n(n+1)}{2}$；三角 $\frac{n(n+1)}{2}+1$；三对角 $3n-2$。



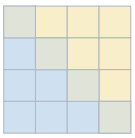
A：10 个



B：10+1 个



C：10 个，不是 12



D：上、下各 10 个

（以 $n=4$ 为例，$\frac{4\times5}{2}=10$，$3\times4-2=10$。）**A、B 对。C 错：**应为 $3n-2$（$n=4$ 时 10 个而非 12 个）。**D 错：**上三角与下三角（都含对角线）个数相同。

✓ **一句话总结：**记住三个数：$\frac{n(n+1)}{2}$、$+1$、$3n-2$。

A 基础巩固 · 填空题

第 16 题（笔记例题）　答案：2016，2019　基础 · 一维地址

考点：$\mathrm{Loc}(i)=a+iL$；一个元素占 L 个连续单元。

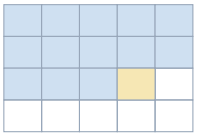
过程：$\mathrm{Loc}(4)=2000+4\times4=2016$，占 2016、2017、2018、2019 共 4 个字节。**易错：**写成 2016–2020（笔记原写法，见纠错 1）——末地址是 $\mathrm{Loc}(i)+L-1$。

✓ **一句话总结：**起址 $a+iL$，末址再加 $L-1$。

第 17 题　答案：1026　基础 · 二维地址

考点：行序：$\mathrm{Loc}(i,j)=\mathrm{Loc}(0,0)+(ni+j)L$。

过程：列数 $n=5$；前面元素 $5\times2+3=13$ 个；$1000+13\times2=1026$。



前面 13 个（蓝色）$\times2=26$

✓ **一句话总结：**二维地址：数格子，再乘 L。

第 18 题　答案：55，12　基础 · 对称矩阵

考点：$\frac{n(n+1)}{2}$；$k=\frac{i(i-1)}{2}+j-1\ (i\geq j)$。

过程：$\frac{10\times11}{2}=55$；$a_{5,3}$：$i\geq j$，$k=\frac{5\times4}{2}+3-1=12$。**检查：**前 4 行有 $1+2+3+4=10$ 个，第 5 行的 a_{51},a_{52} 在前，共 12 个，sa 从 0 开始，所以下标是 12。

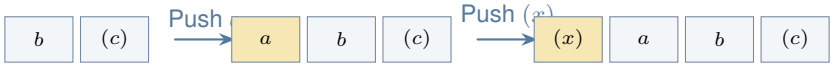
✓ **一句话总结：**k = 前面元素个数（从 0 开始时）。

第 19 题　答案：2，∞；$((x),a,b,(c))$，4　基础 · 广义表性质与操作

考点：递归表；Push 把原子或子表作为新的表头。

(1) $E=(a,E)$ 最外层只有 a 和 E 两个元素，长度 2；展开后 $(a,(a,(a,\cdots)))$ 括号无穷嵌套，深度 ∞。

(2) $L=(b,(c))\xrightarrow{\text{Push}(a)}(a,b,(c))\xrightarrow{\text{Push}((x))}((x),a,b,(c))$，长度 4。**易错：**后压入的在最前面；压入子表 (x) 时，它整体作为一个元素，不能拆成 x。



✓ **一句话总结：**递归表长度有限、深度无穷；Push 永远加在最前面。

第 20 题　答案：8，0.19　基础 · 稀疏因子

考点：$\delta=\frac{t}{m\times n}$。

过程：非零元 22, 15, 11, 17, −6, 39, 91, 28 共 8 个；$\delta=\frac{8}{6\times7}=\frac{4}{21}\approx0.19$。**说明：**$0.19>0.05$，按经验标准并不稀疏，课件只是用这个小矩阵演示方法（见“原始资料重编码与纠错”中的补充说明）。

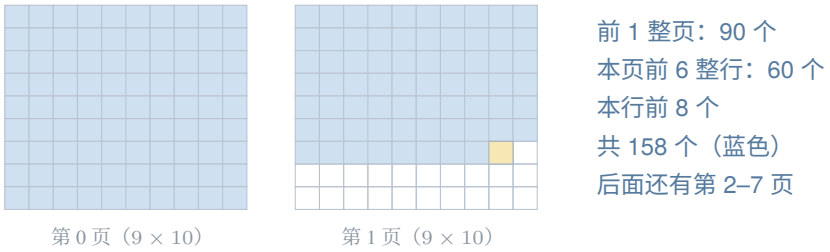
✓ **一句话总结：**δ = 非零元个数 / 元素总数。

A 基础巩固 · 解答题

第 21 题（作业 5.1）　答案：1316；1440　基础 · 三维地址

题意：三维数组，下标都从 0 开始，各维长度 $m_1=8$、$m_2=9$、$m_3=10$；按行序，求某元素地址和总空间。

已知：$a=1000$，$L=2$。　**知识定位：**$\text{Loc}(i_1,i_2,i_3)=a+(i_1m_2m_3+i_2m_3+i_3)L$。



第一步：前面元素个数 $=1\times9\times10+6\times10+8=90+60+8=158$。

第二步：$\text{Loc}(1,6,8)=1000+158\times2=1316$。

第三步：元素总数 $8\times9\times10=720$，占 $720\times2=1440$ 个单元（地址 1000–2439）。

结果检查：最后一个元素 $a_{7,8,9}$ 前面有 719 个，地址 $1000+1438=2438$，占到 2439，与 1440 个单元一致。✓

易错点：把维界 0 : 7 当成 7 个（应为 8 个）；漏乘 L。

同类题怎么做：先写出每一维的长度 $d-c+1$，再按“页—行—列”数前面的元素。

第 22 题（课件第 58 页 / 作业 5.2）　答案：见下　基础 · 三元组表

思路：先写总体信息（行数，列数，非零元个数），再逐行从左到右扫描非零元。



行	列	值
4	5	6
1	1	15
1	5	16
2	2	6
3	3	8
4	1	9
4	4	18

总体信息

答案：总体信息 $(4,5,6)$；三元组 $(1,1,15),(1,5,16),(2,2,6),(3,3,8),(4,1,9),(4,4,18)$。
检查：6 个三元组与总体信息中的 6 一致；行号非递减，同行列号递增。✓
易错点：漏写总体信息；把行列号写成从 0 开始（课件约定从 1 开始）；把同一行的元素顺序写反。

第 23 题　答案：见下　基础 · 三元组还原

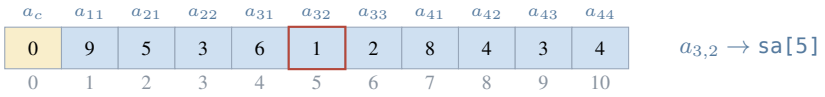
(1) 还原：按总体信息先画 4 行 5 列的全 0 矩阵，再逐个填入：

$$\begin{pmatrix} 0 & 7 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 \\ 4 & 0 & 6 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \end{pmatrix}$$

(2) 有序性：行号依次为 1, 2, 3, 3, 4，非递减；行号同为 3 的两个三元组列号为 1, 3，递增。所以是按行序有序的。
检查：共填入 5 个非零元，与总体信息相符。✓　**易错点：**第 2 行的 -1 在第 5 列，若不看总体信息可能误以为矩阵只有 4 列。

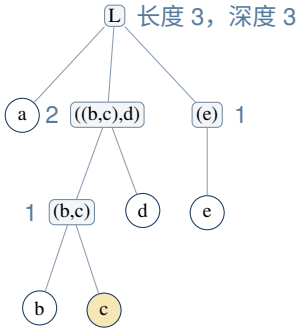
第 24 题　答案：见下　基础 · 下三角矩阵

知识定位：课件约定 $k=\frac{i(i-1)}{2}+j\ (i\ge j)$，$i<j$ 时 $k=0$。



(1) sa $=[0,\ 9,\ 5,\ 3,\ 6,\ 1,\ 2,\ 8,\ 4,\ 3,\ 4]$（sa[0] 是常数 0）。
(2) $a_{3,2}$：$k=\frac{3\times2}{2}+2=5$，sa[5] $=1$；$a_{2,4}$：$i<j$，在常数区，$k=0$，值为 $c=0$。
(3) $\frac{4\times5}{2}+1=11$ 个。
与课程代码对照：下三角矩阵.cpp 的 operator() 正是 elems[row*(row-1)/2+col],对 $(3,2)$ 返回 elems[5]。

第 25 题　答案：见下　基础 · 广义表综合



(1) $\text{head}(L)=a$；$\text{tail}(L)=(((b,c),d),(e))$。

(2) 长度 3；深度 $1+\max(0,2,1)=3$。

(3) $\text{tail}(L)=(((b,c),d),(e))\to\text{head}=((b,c),d)\to\text{head}=(b,c)\to\text{tail}=(c)\to\text{head}=c$，即 $\boxed{c=\text{head}(\text{tail}(\text{head}(\text{head}(\text{tail}(L)))))}$。

检查：第 3.3 节的 C++ 小程序对这串操作输出 c。✓　**易错点：**$\text{tail}((b,c))=(c)$ 不是 c，还要再取一次 head。

B 中等偏下提升 · 单项选择题

第 26 题　答案：B　提升 · 行序与列序

考点：行序偏移 $ni+j$，列序偏移 $mj+i$。

第一步：行序时 $a[2][3]$ 偏移 $=5\times2+3=13$。**第二步：**列序时找 $4j+i=13$，$0\le i\le3$：$j=3,\ i=1$，即 $a[1][3]$。

列序：格中数字 = 偏移

0	4	8	12	16
1	5	9	13 B	17
2	6	10	14 D	18
3	7 C	11 A	15	19

列序偏移：
A $a[3][2]$：11；B $a[1][3]$：13 ✓
C $a[3][1]$：7；D $a[2][3]$：14

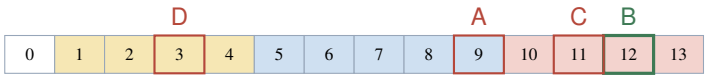
本题最容易卡在：误以为“同一个元素在两种存储下位置一样”而选 D；实际上只有特殊位置（如 $a[0][0]$）才会相同。

✓ **一句话总结：**设未知元素 (i,j)，列出“偏移相等”的方程。

第 27 题　答案：B　提升 · 读 C++ 代码

考点：课程代码的三对角“按对角线”映射。

过程：`row-col` $= 4-3=1$，进入第一个分支：`elems[2*order+row-2]` = `elems[2*5+4-2]` = `elems[12]`。



黄：高对角线 $a_{12}..a_{45}$
蓝：主对角线 $a_{11}..a_{55}$
粉：低对角线 $a_{21}..a_{54}$

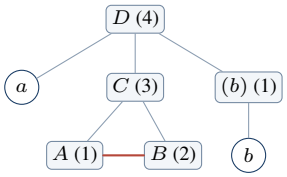
A（9）是课件“按行映射”$2(i-1)+j$ 的结果——映射规则用错了；**C（11）**是 $a_{3,2}$；**D（3）**是高对角线上的 $a_{3,4}$。

✓ **一句话总结：**读代码题严格按代码算，不要套别的公式。

第 28 题　答案：A　提升 · 共享与深度

考点：共享子表代入后求长度、深度。

代入：$A=()$, $B=((\,))$, $C=((\,),((\,)))$, $D=(a,((\,),((\,))),(b))$。**长度：**最外层 a、C、(b) 共 3 个（C 选项 5 是把 C 拆开数了）。**深度：**$\mathrm{Depth}(A)=1$, $\mathrm{Depth}(B)=2$, $\mathrm{Depth}(C)=1+\max(1,2)=3$, $\mathrm{Depth}(D)=1+\max(0,3,1)=4$。



A 同时是 B 和 C 的元素（红线）——共享；
括号中的数是该表的深度；
最长路径 $D \to C \to B \to A$ 经过 4 个表

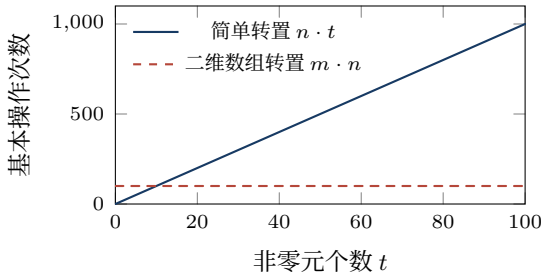
✓ **一句话总结：**共享的子表要“代入展开”后再数，深度看最长的表链。

B 中等偏下提升 · 多项选择题

第 29 题　答案：AB　提升 · 简单转置

考点：`SimpleTranspose` 的循环结构与复杂度。

A 对：外层按列号循环 n 次，内层每次扫描 t 个三元组，$O(n\cdot t)$。**B 对：**按原列号 $1,2,\ldots$ 依次取出，原列号就是新行号；同一列内按原行号递增找到，于是新表同行的列号也递增（图 2.14）。**C 错：**$t\approx mn$ 时为 $O(n\cdot mn)$，比直接转置二维数组的 $O(mn)$ 慢。**D 错：**外层循环变量是 `col`，取 $1\sim$`source.cols`，即原矩阵的**列号**。



$m = n = 10$ 时的比较：$t < 10$ 时简单转置更省，t 越大越不划算

✓ **一句话总结：**三元组方法只在“真稀疏”时划算。

第 30 题　答案：ABD　提升・SetElem

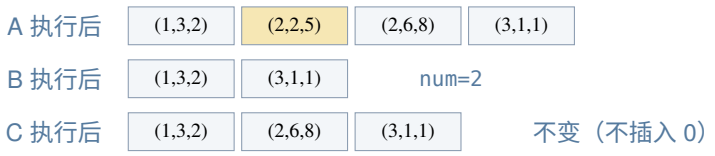
考点：`SetElem` 的修改 / 删除 / 插入三种情况与越界检查。

A 对：从下标 2 往前找：$(2,2)$ 排在 $(3,1)$、$(2,6)$ 之前，排在 $(1,3)$ 之后，停在 `pos=0`，插入到 `pos+1=1`，后两个三元组后移（图 2.13）。

B 对：找到 $(2,6)$ 且 $v=0$，删除并 `num--`。

C 错：未找到且 $v=0$，代码最后一个分支“什么也不做”，直接返回 `true`——三元组表永远不存 0。

D 对：`r=7 > rows=5`，第一个 `if` 返回 `false`（`07_sparse.cpp` 实测输出 0）。



✓ **一句话总结：**SetElem：找到改 / 删，找不到插；0 永不入表。

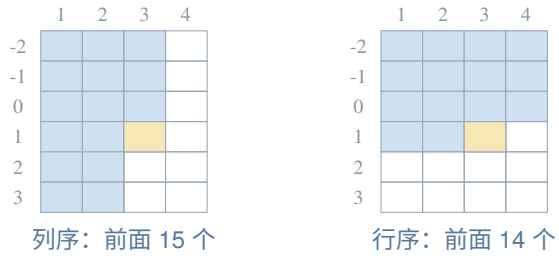
B 中等偏下提升・填空题

第 31 题　答案：560，556　提升・一般下标

列数、行数：行 $-2 \sim 3$ 共 $m = 6$ 行；列 $1 \sim 4$ 共 $n = 4$ 列。

列序：$[(d_1 - c_1 + 1)(j - c_2) + (i - c_1)]L = [6 \times (3-1) + (1+2)] \times 4 = 15 \times 4 = 60$，地址 560。

行序：$[(d_2 - c_2 + 1)(i - c_1) + (j - c_2)]L = [4 \times 3 + 2] \times 4 = 56$，地址 556。



✓ **一句话总结：**先写出行数、列数，再决定乘哪一个。

第 32 题　答案：4，24，120，328　提升・n 维地址

过程：$b_1 = 4,\ b_2 = 5,\ b_3 = 6,\ L = 4$。$c_3 = L = 4$；$c_2 = b_3 c_3 = 24$；$c_1 = b_2 c_2 = 120$。偏移 $= 120 \times 2 + 24 \times 3 + 4 \times 4 = 240 + 72 + 16 = 328$。

含义：第 1 维下标加 1 要跳过 $5 \times 6 = 30$ 个 int（120 字节）；第 2 维加 1 跳过 6 个 int（24 字节）。**检查：**前面元素 $2 \times 30 + 3 \times 6 + 4 = 82$ 个，$82 \times 4 = 328$。✓

✓ **一句话总结：**c_i = 第 i 维下标加 1 时地址的增量。

第 33 题　答案：14，16　提升 · 上三角

过程： $a_{4,5}$：$\frac{(4-1)(10-4+2)}{2}+5-4+1=12+2=14$。$a_{4,1}$：$4>1$，在常数区，$B[\frac{5\times6}{2}+1]=B[16]$。

1	2	3	4	5
16	6	7	8	9
16	16	10	11	12
16	16	16	13	14
16	16	16	16	15

格中数字 = 在 B 中的位置；
每行个数 5, 4, 3, 2, 1；
下方米黄全部共用 $B[16]$

✓ **一句话总结：** 上三角第 r 行有 $n-r+1$ 个，本行前有 $j-i$ 个。

B 中等偏下提升 · 综合大题

第 34 题　答案：见下　提升 · 三元组全流程

1. 题目在问什么： 把同一个稀疏矩阵依次做“存储 → 转置 → 相乘”，每一步都要保持三元组表按行有序且只存非零元。

2. 知识链： 三元组表的建立 → 简单转置（按原列号扫描）→ 乘法公式 → “只存非零结果”。

(1) A 是 3 行 4 列，非零元 $3,7,-1,2$：$(3,4,4)$；$(1,1,3),(1,4,7),(2,4,-1),(3,2,2)$。

(2) `dest` 的总体信息为 $(4,3,4)$。外层循环：

`col`	在 `source` 中找到	存入 `dest`（行列互换）
1	$(1,1,3)$	`dest[0]` $=(1,1,3)$
2	$(3,2,2)$	`dest[1]` $=(2,3,2)$
3	无	—
4	$(1,4,7)$，$(2,4,-1)$	`dest[2]` $=(4,1,7)$，`dest[3]` $=(4,2,-1)$

A^{T}：$(4,3,4)$；$(1,1,3),(2,3,2),(4,1,7),(4,2,-1)$——与 `07_sparse.cpp` 输出一致。

(3) C 为 3×2。$c_{12}=3\times1+0\times0+0\times4+7\times0=3$；$c_{31}=0\times0+2\times2+0\times3+0\times0=4$；其余为 0。$C$ 的三元组表：$(3,2,2)$；$(1,2,3),(3,1,4)$。

(4) $c_{11}=3\times0+0\times2+0\times3+7\times0=0$：$A$ 第 1 行的两个非零元 3、7 分别遇到了 B 第 1 列中的 0，**没有出现非零乘积**，结果为 0。按“只有积矩阵元素不为 0 时才添加”的规则，不存入三元组表。

$$\begin{pmatrix}3&0&0&7\\0&0&0&-1\\0&2&0&0\end{pmatrix}\times\begin{pmatrix}0&1\\2&0\\3&4\\0&0\end{pmatrix}$$

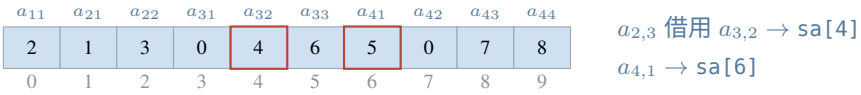
c_{11}：黄色（A 第 1 行非零）只与蓝色（B 第 1 列）相乘，对应位置 B 恰好都是 0 $\Rightarrow c_{11}=0$，不入表

3. 合理性检查： C 的三元组表行号 $1,3$ 递增；C 中没有存任何 0。✓
4. 最容易卡在： 转置时直接对调得到 $(1,1,3),(4,1,7),(4,2,-1),(2,3,2)$，行号不递增。
5. 变式思考： 若把 B 的 b_{41} 改成 1，c_{11} 会变成 7，此时 C 的三元组表在最前面多出 $(1,1,7)$，总体信息变为 $(3,2,3)$。

第 35 题

答案：见下　提升 · 对称矩阵 + C++

1. 题目在问什么： 把对称矩阵压缩，再用公式（及其 C++ 实现）查元素，最后算节省的空间。
(1) 按行取下三角：`sa` $=[2,\ 1,3,\ 0,4,6,\ 5,0,7,8]$（共 $\frac{4\times5}{2}=10$ 个）。
(2) $a_{2,3}$：$i<j$，交换，$k=\frac{3\times2}{2}+2-1=4$，`sa[4]` $=4$（原矩阵 $a_{23}=4$ ✓）；$a_{4,1}$：$i\geq j$，$k=\frac{4\times3}{2}+1-1=6$，`sa[6]` $=5$（原矩阵 $a_{41}=5$ ✓）。



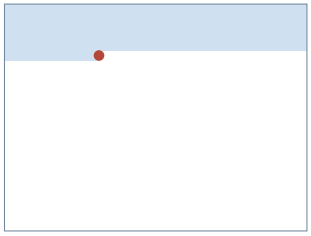
(3) 空 (1)：`i * (i - 1) / 2 + j - 1;`　空 (2)：`j * (j - 1) / 2 + i - 1`。（与课程代码 对称矩阵.cpp 一致；注意先乘后除。）
(4) 常规存储 $100\times100\times4=40000$ 字节；压缩 $5050\times4=20200$ 字节；节省 19800 字节（约一半）。
最容易卡在： 空 (2) 处忘记交换 i,j；或写成 `(i-1)/2*i`，先除后乘造成整数截断。
变式思考： 若改为只存上三角、按行存入 `sa[0..]`，则 $i\leq j$ 时 $k=\frac{(i-1)(2n-i+2)}{2}+j-i$，$i>j$ 时交换 i,j。

C 跨学科迁移 · 详细解析

第 36 题

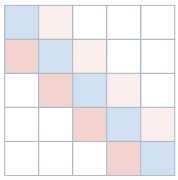
答案：A　跨学科 · 单选

当前专题知识： 二维数组行序地址公式。**借用的外部情境：** 数字图像处理中的灰度图存储。**为什么不算超纲：** 题干已给出“每像素 1 字节”“按行存放”“`img[480][640]`”，不需要任何图像处理知识。**迁移点：** “行 × 列”的像素阵列就是二维数组，像素坐标就是下标。
计算： 列数 $n=640$，偏移 $=640\times100+200=64200$。**B（96100）** $=480\times200+100$，是列序结果；**C（64000）**漏加列下标 200；**D（128400）**多乘了 2（误以为每像素 2 字节）。



第 37 题　答案：ABC　跨学科 · 多选

当前专题知识：三对角矩阵、对称矩阵的压缩存储。**借用的外部情境：**电路中的节点分析（链式电阻网络）。**为什么不算超纲：**题干直接给出了 G 的非零分布和对称性，不需要会列节点方程。**迁移点：**“只与相邻节点相连”⇔ 三对角；“相互作用对等”⇔ 对称。

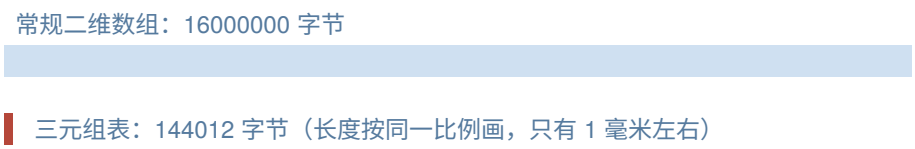


蓝：主对角线 n 个；深粉：下次对角线 $n-1$ 个；
浅粉：上次对角线，与下次对角线对称相等，不必再存
⇒ 只存 $n+(n-1)=2n-1$ 个（选项 C）

A 对：题干给出的分布即 $|i-j|\le 1$。**B 对：**$g_{ij}=g_{ji}$，可按对称矩阵存下三角 $\frac{n(n+1)}{2}$ 个（虽然这样存了很多 0，但方法本身正确）。**C 对：**见上图。**D 错：**应为 $3n-2=2998$。

第 38 题　答案：144012；16000000；0.003　跨学科 · 填空

当前专题知识：稀疏矩阵的三元组存储与稀疏因子。**借用的外部情境：**社交网络的“关注”关系矩阵（第 7 章“图”的邻接矩阵）。**为什么不算超纲：**只用到矩阵的大小和非零元个数，题干都已给出。**迁移点：**“用户—用户”关系表就是稀疏矩阵，一条关注 = 一个三元组。**(1)** $(12000+1)\times 3\times 4=144012$ 字节。**(2)** $2000\times 2000\times 4=16000000$ 字节（约 16 MB）。**(3)** $\delta=\frac{12000}{4000000}=0.003\le 0.05$，是真正的稀疏矩阵；三元组表只需常规存储约 0.9% 的空间。



考前总复习

高频易错点总表

易错点	错误做法	正确做法
行序公式乘谁	$\mathrm{Loc}(i,j)=\mathrm{Loc}(0,0)+(mi+j)L$	行序乘**列数** n；列序乘**行数** m
一般下标的个数	下标 $2\sim 6$ 算成 4 个	个数 $=d-c+1=5$；偏移 $=i-c$
漏乘 L	三维公式写成 $a+i_1m_2m_3+i_2m_3+i_3$	整个“前面元素个数”乘 L
地址范围	`a[4]` 占 2016–2020	末地址 = 起址 $+L-1$
对称矩阵 $i<j$	直接代 $\frac{i(i-1)}{2}+j-1$	先交换 i,j
下标从几开始	0 起和 1 起的公式混用	先确认 `sa` / B 的起点和常数 c 的位置（表 2.1）
上三角本行偏移	本行前有 $j-1$ 个	本行从 a_{ii} 开始，前有 $j-i$ 个
三对角映射	读代码题套课件公式	课件按行：$2(i-1)+j$；课程代码按对角线：三个分支
三元组表	漏写总体信息；存入 0	首行写（行数，列数，非零元个数）；0 不入表
稀疏转置	只对调行列号	按原列号逐列扫描，保证新表按行有序
表尾	$\mathrm{tail}((a,b,c))=c$	表尾是表：(b,c)；单元素表的表尾是 $()$
表头	$\mathrm{head}((a,b))=(a)$	表头是元素本身：a
长度 / 深度	长度数所有原子；空表深度 0	长度只数最外层；原子 0、空表 1
C++ 写法	`int a[n];`（n 是变量）；先除后乘	长度用常量或 `new`；`i*(i-1)/2` 先乘后除

基础解题步骤总结

✓ 四类核心题型的固定步骤

1. 数组地址题：① 写出每一维的个数 $d-c+1$；② 确认行序还是列序；③ 数“前面有几个元素”（整页 / 整行 / 本行）；④ 乘 L 加基址；⑤ 用第一个或最后一个元素检验。

2. 特殊矩阵压缩题：① 确认存哪一部分（上 / 下三角、带内）；② 确认 `sa` 从 0 还是从 1 开

始、常数放哪；③ 数前面的元素（阶梯 / 每行 $n-r+1$ 个 / 每行 3 个）；④ 代 a_{11} 和最后一个元素检验公式。

3. 稀疏矩阵题： ① 先写总体信息；② 逐行从左到右写三元组；③ 转置按原列号逐列扫描；④ 加、乘只存非零结果；⑤ 核对三元组个数与总体信息。

4. 广义表题： ① 先圈出最外层的元素（算长度）；② 表头取元素本身，表尾加括号；③ 深度用“自内向外 $1+\max$”或“数括号”两种方法互相验证；④ 取原子时每一步都写出中间结果。

相近概念与跨学科 / 相似知识总对比

本章知识	对照对象	可以直接搬的（✓）	要加条件或不能搬的（△/✗）
数组	第 2 章顺序表	顺序存储、随机存取、地址 = 基址 + 前面个数 $\times L$	✗ 插入删除；△ 下标起点
行序地址	进位计数制	位值 $\times$ 权再求和	✗ 第一维长度不影响偏移；✗ 非 0 起点要先平移
C++ 行序	MATLAB / FORTRAN 列序	地址是下标的线性函数	✗ 相邻关系不同；△ MATLAB 从 1 开始
对称矩阵压缩	线性代数对称矩阵	和、数乘仍对称，可压缩	✗ 乘积一般不对称
三角矩阵压缩	线性代数三角矩阵	转置后变成另一种三角	△ 乘积 / 行列式性质只在 $c=0$ 时成立
三对角矩阵	弹簧链 / 电阻链（拓展）	只存带内元素	△ 只有作用对等时才对称
三元组 / 行指针链表	第 7 章图的邻接表	零元不存；无向图可按对称压缩	✗ 有向图不能按对称压缩
广义表	线性表	直接元素之间线性	✗ 长度 $\neq$ 原子个数；✗ 表尾不是最后一个元素
广义表深度	第 3 章栈的括号匹配	遇“(”入栈、“)”出栈	△ 需去掉表名；✗ 递归表深度 ∞

后续课程连接

- **第 6 章树与二叉树：** 不共享、不递归的广义表就是一棵树（笔记最后一页写下了这个连接）；树的“深度”与广义表的深度思想相近，但计数起点要按第 6 章的定义。
- **第 7 章图：** 邻接矩阵是二维数组；无向图的邻接矩阵可按对称矩阵压缩；稀疏图用邻接表（= 带行指针的链表的思想）。
- **数值计算 / 自动控制：** MATLAB 列优先存储、稀疏矩阵运算、三对角方程组的追赶法。
- **图像处理 / 计算机视觉：** OpenCV 等库中图像按行连续存放，像素寻址就是二维地址公式。

随附 C++ 源文件

文件	内容（全部用 `g++ -std=c++11 -Wall` 编译运行通过）
`01_address.cpp`	用指针验证二维、三维数组的行优先地址公式
`02_typedef.cpp`	用 `typedef` 声明“数组的数组”
`03_matrix.cpp`	课件 Matrix 类的 `operator()` 映射
`04_symmetric.cpp`	对称矩阵压缩与全位置校验
`05_triangular.cpp`	下三角（课件约定）与上三角（笔记约定）公式校验
`06_tridiagonal.cpp`	三对角矩阵两种映射对照
`07_sparse.cpp + TriSparseMatrix.h`	课程三元组类：SetElem 插入 / 删除、简单转置、乘法
`08_glist.cpp`	用字符串演示广义表的表头、表尾、长度、深度
`09_check.cpp、10_check_glist.cpp`	练习题数值答案的统一校验

参考资料

原始教学依据：某高校相关学院　　《数据结构与算法》课件“05 第 5 章: 数组和广义表”（80 页）；课堂笔记（2026 年 10 月 19–20 日，9 页）；课程配套 C++ 程序（对称矩阵、下三角矩阵、三对角矩阵、稀疏矩阵三元组顺序表）。

核心定义与公式核验：严蔚敏、吴伟民《数据结构（C 语言版）》第 5 章（数组的顺序表示、矩阵的压缩存储、广义表的定义）；行优先 / 列优先地址公式另参照 Wikipedia “Row- and column-major order”条目（C/C++、Pascal 行优先，Fortran 列优先）。所有地址与下标公式均已用随附 C++ 程序逐一验证。

学科史资料：Wikipedia “Harry Markowitz”（1989 年冯·诺依曼理论奖，表彰其包括稀疏矩阵方法在内的贡献）；Wikipedia “CAR and CDR”及 J. McCarthy, *Recursive Functions of Symbolic Expressions and Their Computation by Machine*（1960）; Fortran 数组存储顺序说明（Fortran Programming Guide, Array Storage Order）。